

ОДЕССКИЙ КОРПОРАТИВНЫЙ КОМПЬЮТЕРНЫЙ КОЛЛЕДЖ

Проверили:

Содержание: Славкина Виктория Владимировна

Оценка: _____ Подпись: _____

Оформление: Рудниченко Николай Дмитриевич

Оценка: _____ Подпись: _____

ДИПЛОМНЫЙ ПРОЕКТ

На тему:

«C++ *RPG FARMER*»

Ученик группы 46-4С/24

Гнапа Максима Сергеевич

Одесса, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1. ПОСТАНОВКА ЗАДАЧИ	6
2. ОБОСНОВАНИЕ ВЫБОРА ЯЗЫКОВ И ТЕХНОЛОГИЙ	8
2.1 C++	8
2.2 JSON	12
2.3 Unreal Engine 5.....	15
2.4 Visual Studio 2022	18
2.5 Blueprint и assets Unreal Engine	21
2.6 Unreal Build Tool и система сборки проекта	24
2.7 Git и контроль версий	26
3. ПРОЦЕСС ПРОЕКТИРОВАНИЯ В СРЕДЕ РАЗРАБОТКИ.....	30
3.1 Описание порядка использования Visual Studio и Unreal Engine 5	30
3.2 Структура проекта в Visual Studio.....	31
3.3 Работа с исходным кодом проекта	31
3.4 Работа с ресурсами и assets	33
3.5 Конфигурационные файлы проекта	34
3.6 Средства проверки и отладки	36
3.7 Рабочие файлы и сохранения	37
4. ЭТАПЫ РАЗРАБОТКИ	40
4.1 Создание проекта приложения	40
4.2 Создание персонажа.....	43
4.3 Добавление ходьбы, управления камерой и анимаций	49
4.4 Реализация управления, камеры и системы stamina.....	54
4.5 Создание рудных объектов	59
4.6 Добыча ресурсов и интерфейс	63
4.7 Добавление возможности прокачки персонажа.....	66

4.8 Инвентарь, зелья и меню персонажа.....	70
4.9 Торговец и торговля ресурсами.....	73
4.10 Мини-задания, рабочие заказы и развитие игрового цикла	76
4.11 Сохранение прогресса	80
5. ПРОВЕРКА РАБОТЫ ПРОГРАММЫ.....	85
ЗАКЛЮЧЕНИЕ	92
СПИСОК ЛИТЕРАТУРЫ	94
ДОДАТОК А	96

ВВЕДЕНИЕ

Современные игровые проекты всё чаще строятся не только вокруг визуальной части, но и вокруг понятного игрового цикла, который удерживает внимание пользователя. Даже небольшой учебный проект становится значительно интереснее, если в нём есть цель, прогресс, награда и возможность улучшать персонажа. Дипломный проект «*C++ RPG FARMER*» посвящён разработке такой игровой системы на базе *Unreal Engine 5* и языка *C++*.

Разрабатываемая игра относится к жанру небольшого *RPG*-проекта с элементами добычи ресурсов. Игрок управляет персонажем, который перемещается по игровой карте, добывает руду киркой, следит за запасом стамины, продаёт ресурсы торговцу, покупает зелья и постепенно усиливает характеристики персонажа. За счёт этого формируется основной цикл: добыча ресурсов, торговля, получение опыта и прокачка.

Актуальность проекта заключается в том, что он позволяет показать практическое применение объектно-ориентированного программирования, игровых подсистем, пользовательского интерфейса, сохранений и взаимодействия с игровым миром. В проекте используются классы персонажа, контроллера, *HUD*, торговца, рудных объектов, подсистемы рабочих заказов и *JSON*-сохранений. Такой набор компонентов демонстрирует не отдельную лабораторную работу, а связанную игровую программу.

Целью дипломного проекта является создание игровой системы *RPG FARM*, в которой пользователь может управлять персонажем, добывать руду, взаимодействовать с торговцем, выполнять мини-задания, проходить рабочие смены по заказам и сохранять прогресс. Проект также должен иметь понятный интерфейс, чтобы игрок видел количество руды, золота, зелий, уровень, опыт, стамину и активные цели.

В ходе разработки особое внимание уделялось тому, чтобы игровые механики были связаны между собой. Добыча руды даёт ресурсы и опыт, ресурсы можно продать, золото можно потратить на зелья, опыт повышает уровень, уровень даёт очки улучшений, а улучшения делают дальнейшую добычу удобнее. После расширения проекта добавлена система рабочих заказов, благодаря которой игровой процесс получает более чёткую структуру и цель внутри каждой смены. Общий вид проекта *C++ RPG FARMER*

1. ПОСТАНОВКА ЗАДАЧИ

Перед началом разработки была поставлена задача создать учебный игровой проект на C++ с использованием *Unreal Engine 5*. Проект должен демонстрировать базовые принципы разработки игры: создание игрового персонажа, обработку ввода, перемещение по карте, работу камеры, взаимодействие с объектами мира, отображение интерфейса и сохранение данных.

Основная игровая задача заключается в создании цикла «добыча -> продажа -> прокачка». Основной игровой цикл проекта

Игрок должен иметь возможность находить рудные залежи, наносить по ним урон киркой, получать руду после разрушения объекта, продавать добытые ресурсы торговцу и использовать заработанные средства для дальнейшего развития персонажа.

Дополнительной задачей является расширение простого цикла добычи системой целей. Для этого в проект вводятся мини-задания и рабочие заказы. Мини-задания помогают игроку видеть общий прогресс по базовым действиям, а рабочие заказы задают цель на конкретную игровую смену. Такой подход делает игровой процесс более понятным и мотивирует игрока не просто перемещаться по карте, а выполнять конкретные условия.

В проекте необходимо реализовать систему стамины. Персонаж должен иметь ограниченный запас энергии для спринта, а при окончании стамины не должен продолжать ускоренное движение. При этом стамина должна восстанавливаться со временем, а игрок должен иметь возможность использовать зелья для быстрого восстановления запаса энергии.

Также требуется реализовать систему прокачки. Игрок получает опыт за полезные действия, повышает уровень, получает очки улучшений и может тратить их на увеличение максимальной стамины, урона по руде и скорости

передвижения. Базовый урон по руде установлен на уровне 50 единиц, а каждое улучшение урона увеличивает его на 10 единиц.

Отдельное требование связано с пользовательским интерфейсом. Игрок должен видеть текущее количество руды, золота, зелий, уровень, опыт, стамину и подсказки взаимодействия. В меню паузы должны быть доступны просмотр целей, настройки, сброс прогресса, выход в лобби и выход из игры. В настройках должны присутствовать звук, управление, интерфейс и возможность изменения привязок клавиш. Сроки выполнения проекта приведены в таблице 1.1.

Итоговая программа должна быть стабильной, удобной для пользователя и готовой к дальнейшему развитию. Архитектура проекта должна обеспечивать возможность добавления новых типов руды, заказов, характеристик персонажа и элементов интерфейса без необходимости полностью перерабатывать существующий код.

Таблица 1.1 – Сроки выполнения проекта

№ этапа	Срок выполнения	Объём готовности	Комментарий
1	до 10.02.2026	40%	Нач. этап. разработки
2	до 19.03.2026	70%	Основ. функции
3	до 21.04.2026	100%	Завершение
4	до 26.05.2026	-	Исправление

2. ОБОСНОВАНИЕ ВЫБОРА ЯЗЫКОВ И ТЕХНОЛОГИЙ

2.1 C++

Основным языком разработки проекта «C++ RPG FARMER» выбран C++. Этот язык подходит для создания игровой логики в *Unreal Engine 5*, потому что позволяет описывать поведение персонажа, объектов, *controller*, *user interface* и *save system* в виде понятного кода.

C++ относится к компилируемым языкам программирования. Перед запуском исходный код преобразуется в машинный код, поэтому игровые действия выполняются быстро и предсказуемо.

В проекте C++ используется как основа архитектуры. Он помогает разделить игровые системы на отдельные классы и методы, благодаря чему код проще проверять, изменять и расширять.

Одним из главных преимуществ C++ является поддержка объектно-ориентированного программирования. Для игры это удобно, потому что персонаж, руда, торговец, *controller HUD* и *save layer* могут быть представлены как отдельные объекты.

Класс *AGameplayCharacterBase* используется для общей логики игрового персонажа. В нём удобно хранить характеристики и действия, связанные с ресурсами, *stamina*, опытом и развитием персонажа.

Класс *AThiefPlayerController* отвечает за управление игроком и переключение игровых состояний. Такой подход отделяет ввод и меню от логики самого персонажа.

Класс *APlayerGameHUD* отвечает за отображение *user interface*. Через него игрок видит ресурсы, *stamina*, опыт, подсказки, меню, торговлю и другие игровые окна.

Класс *AOreStoneBase* используется для рудных объектов. Он позволяет хранить состояние руды, принимать урон и выдавать игроку ресурс после разрушения объекта.

Класс *ATraderNPC* отвечает за торговца. Он используется как точка взаимодействия, через которую игрок может продавать руду, покупать зелья и работать с заказами.

Отдельную роль играет *UWorkOrderSubsystem*. Такая *subsystem* удобна для хранения заказов, целей и наград отдельно от конкретных объектов на карте.

Выбор C++ обоснован следующими преимуществами:

- высокая производительность, C++ позволяет быстро выполнять игровую логику;
- компиляция в машинный код, программа заранее преобразуется в команды для процессора;
- поддержка объектно-ориентированной архитектуры, проект можно разделить на классы с разной ответственностью;
- удобная интеграция с *Unreal Engine 5*, язык является нативным для движка;
- строгая типизация, ошибки легче находить ещё на этапе компиляции;
- точное управление игровой логикой, условия действий можно описывать явно;
- масштабируемость, новые механики можно добавлять без полной переработки проекта.

Важным преимуществом C++ является возможность связывать игровые системы между собой. Например, добыча руды может изменять ресурсы игрока, обновлять *HUD* и влиять на прогресс заданий.

В проекте активно используются методы классов. Метод показывает, какое действие выполняет объект, поэтому код становится понятнее и удобнее для сопровождения.

C++ также позволяет использовать перечисления. Они нужны там, где значение выбирается из заранее заданного набора, например для улучшений, меню, целей или состояний игры.

Основные принципы объектно-ориентированного программирования проявляются следующим образом:

- инкапсуляция, данные и методы объединяются внутри классов;
- наследование, игровые классы создаются на основе базовых классов *Unreal Engine*;
- разделение ответственности, каждая система отвечает за свою часть проекта;
- повторное использование кода, общую логику можно вызывать из разных мест;
- расширяемость, новые механики можно добавлять через классы, методы и *subsystem*.

Для *Unreal Engine* язык C++ важен ещё и потому, что работает с системой отражения движка. Макросы *UCLASS*, *UFUNCTION* и *UPROPERTY* позволяют движку видеть классы, функции и переменные.

В проекте это помогает связать код и визуальные *assets*. Основная логика остаётся в C++, а модели, анимации и отдельные настройки могут подключаться через *Blueprint*.

C++ также удобен для настройки игрового баланса. Числовые параметры, связанные с уроном, *stamina*, скоростью, стоимостью зелий и опытом, можно хранить и изменять в коде.

Особенно важна роль C++ в системе *stamina*. Код позволяет точно проверять, когда персонаж ускоряется, когда тратит энергию и когда должен вернуться к обычному движению.

Ещё один пример применения C++ связан с рабочими заказами. Отдельная *subsystem* позволяет хранить цели, проверять прогресс и выдавать награды без смешивания этой логики с *user interface*.

В проекте C++ связывает несколько уровней программы. Персонаж выполняет действия, *controller* принимает ввод, *HUD* отображает состояние, а игровые системы проверяют прогресс. Для учебного проекта C++ полезен тем, что показывает реальные навыки разработки. В работе используются классы, методы, поля, структуры, условия, сохранения и взаимодействие с *API* движка. Основные C++ классы проекта приведены на рисунке 2.1.

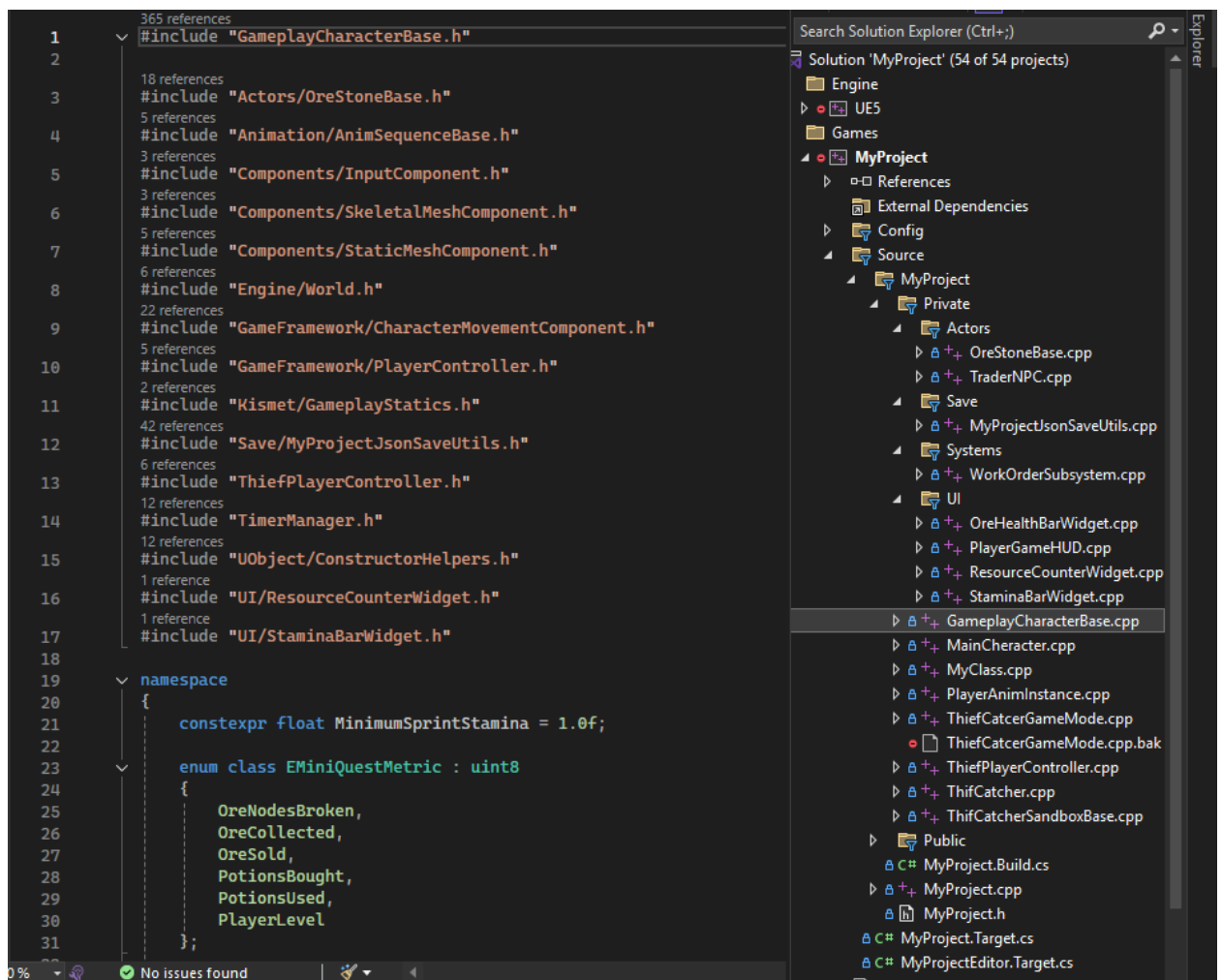


Рисунок 2.1 - Основные C++ классы проекта

Таким образом, C++ выбран потому, что он подходит для основной игровой логики в *Unreal Engine 5*. Он обеспечивает производительность, строгую структуру, объектно-ориентированный подход и возможность дальнейшего развития проекта.

2.2 JSON

Для хранения данных в проекте выбран формат *JSON*. Полное название этого формата – *JavaScript Object Notation*, но он не ограничивается языком *JavaScript* и используется в разных типах программ.

Выбор *JSON* связан с тем, что проект содержит постоянный прогресс. Игрок добывает руду, получает золото, покупает зелья, повышает уровень, улучшает характеристики, выполняет мини-задания и проходит рабочие смены.

Главное преимущество *JSON* заключается в читаемости. Данные записываются в обычный текстовый файл, который можно открыть через любой текстовый редактор.

В отличие от бинарных файлов, *JSON* легко понять визуально. Каждое значение имеет название, поэтому разработчик может быстро определить, за что отвечает конкретная строка.

В проекте *C++ RPG FARMER* сохранение данных связано с отдельным слоем чтения и записи файлов. Такой подход отделяет игровые действия от сохранения и загрузки данных.

Структура *JSON* строится на парах ключ и значение. Ключ обозначает название параметра, а значение показывает сохранённые данные.

В проекте сохраняются разные типы данных. Целые числа используются для руды, золота, зелий и уровня, дробные числа подходят для *stamina* и скорости, а строки могут хранить язык *interface* или клавиши управления.

В проекте удобно сохранять следующие данные:

- количество добытой руды показывает текущий ресурс игрока;
- количество золота отражает результат торговли и наград;
- количество зелий *stamina* сохраняет инвентарь расходников;
- текущий уровень персонажа показывает общий прогресс;

- текущее количество опыта показывает приближение к следующему уровню;
- количество очков улучшений показывает доступные усиления;
- максимальная *stamina* хранит улучшенное значение энергии;
- урон по руде сохраняет текущую силу удара;
- скорость передвижения хранит результат улучшения мобильности;
- статистика добычи, торговли и зелий помогает отслеживать задания.

Отдельное значение имеет сохранение настроек. Игрок может изменять громкость, чувствительность управления, язык *interface*, масштаб меню и привязки клавиш.

Настройки также удобно хранить в формате *JSON*. Отдельные поля могут отвечать за громкость, музыку, чувствительность мыши, клавиши атаки, взаимодействия, торговли, прокачки и использования зелья.

Формат *JSON* полезен не только для игрока, но и для разработчика. Во время тестирования можно быстро проверить, правильно ли изменились ресурсы, золото, зелья, опыт и уровень.

Также *JSON* помогает проверять сброс прогресса. После сброса ресурсы, золото, зелья, уровень, опыт, улучшения, статистика и данные заказов должны вернуться к начальным значениям.

Важным преимуществом *JSON* является расширяемость. При развитии проекта в файл можно добавить новые поля, например репутацию, новые ресурсы, завершённые смены или данные заказов.

В проекте мини-задания и рабочие заказы используют статистику действий игрока. Эти данные помогают отслеживать прогресс целей и показывать выполнение задач.

Преимущества формата *JSON* для проекта заключаются в следующем:

- читаемость файла позволяет быстро проверять сохранённые значения;
- простая структура ключ значение делает данные понятными;

- поддержка разных типов данных позволяет хранить числа, строки, логические значения и списки;
- удобство отладки помогает находить ошибки в ресурсах, опыте, уровне и настройках;
- возможность расширения позволяет добавлять новые поля сохранения;
- независимость от языка делает формат удобным для игрового кода и внешних редакторов.

При загрузке данных из файла важно учитывать возможные ошибки. Файл может быть повреждён, удалён или изменён вручную, поэтому игра должна использовать корректные данные или значения по умолчанию.

Также важно проверять допустимость числовых значений. Максимальная *stamina*, количество золота и скорость персонажа должны загружаться в безопасном диапазоне.

Формат *JSON* хорошо подходит для небольшого и среднего объёма данных. В проекте сохраняются параметры персонажа, настройки, статистика и данные рабочих заказов.

Пример файла сохранения в формате JSON приведён на рисунке 2.2.

```
{
  "maxStamina": 100,
  "currentStamina": 100,
  "collectedOreResources": 1,
  "collectedGold": 15,
  "staminaPotionCount": 0,
  "playerLevel": 2,
  "currentExperience": 25,
  "experienceToNextLevel": 150,
  "availableUpgradePoints": 0,
  "oreDamage": 60,
  "walkSpeed": 450,
  "sprintSpeed": 700,
  "totalOreCollected": 5,
  "totalOreSold": 4,
  "totalOreNodesBroken": 5,
  "totalGoldEarned": 0,
  "totalPotionsBought": 1,
  "totalPotionsUsed": 1
}
```

Рисунок 2.2 - Пример файла сохранения в формате JSON

Ограничения у *JSON* тоже есть. Так как файл является текстовым, его можно открыть и изменить вручную, но для учебного проекта это помогает проверять сохранения.

Ещё одно ограничение связано с большими объёмами данных. Для огромной карты, множества объектов и сложного состояния мира могут потребоваться более быстрые форматы, но в данном проекте сохраняются только конкретные параметры игрока и настройки.

Таким образом, формат *JSON* выбран потому, что он простой, читаемый, удобный для отладки и достаточно гибкий. В игре *C++ RPG FARMER* он используется для сохранения прогресса, ресурсов, характеристик, зелий, настроек, мини-заданий и рабочих заказов.

2.3 Unreal Engine 5

В качестве основного *game engine* для проекта *C++ RPG FARMER* выбран *Unreal Engine 5*. Он подходит для разработки трёхмерных игр, потому что объединяет *level editor*, систему игровых объектов, инструменты для персонажей, *animation*, материалы, *collision*, камеру, ввод и *interface*.

Unreal Engine 5 используется не только как визуальный редактор, но и как полноценная среда разработки игры. Внутри него можно создавать *levels*, размещать объекты, подключать модели, настраивать материалы и запускать проект для проверки.

Для проекта *RPG FARM* важна поддержка связки *C++* и *Blueprint*. Основная логика пишется в коде, а визуальные и редакторские настройки удобно выполнять через *assets* движка.

В проекте используется разделение на карту меню и игровую карту. Карта *NewMap* применяется как стартовая, а карта *Showcase* из набора *Namaqualand* используется для проверки основного игрового цикла.

Такое разделение *levels* делает структуру проекта понятнее. Меню отвечает за запуск игры, а основная карта отвечает за игровой процесс.

Одной из ключевых возможностей *Unreal Engine 5* является система игровых объектов. Почти каждый объект на *level* может быть представлен как *Actor*, например персонаж, руда, торговец, свет или интерактивная зона.

Персонаж игрока реализован через классы, связанные с системой *Character*. Логика персонажа, *controller* и *HUD* работают вместе с возможностями движка, поэтому проект строится не только на коде, но и на системах *Unreal Engine*.

Важным преимуществом *Unreal Engine 5* является компонентная система. Объект может состоять из модели, *collision*, камеры и дополнительных компонентов, поэтому его поведение проще разделять на отдельные части.

Основные возможности *Unreal Engine 5*, важные для проекта, включают:

- поддержку трёхмерных *levels*, где можно создавать карту с окружением и рабочими зонами;
- систему объектов *Actor*, которая позволяет размещать на карте игровые сущности;
- систему *Character*, которая упрощает создание управляемого персонажа;
- компонентную архитектуру, которая разделяет модель, *collision*, камеру и игровые элементы;
- поддержку *C++*, благодаря которой основная логика пишется в структурированном виде;
- поддержку *Blueprint*, которая помогает настраивать связи, *assets*, *animation* и события;
- систему *collision*, которая нужна для атаки руды и взаимодействия с объектами;

- систему *animation*, которая позволяет подключать движение, бег и удар киркой;
- встроенный *level editor*, который помогает быстро размещать объекты на карте;
- инструменты *interface*, которые позволяют отображать ресурсы, *stamina*, опыт, меню и настройки.

Для механики добычи руды особенно важны *collision* и проверка расстояния. Когда игрок нажимает кнопку атаки, код определяет, находится ли рудный объект в зоне удара.

Система *animation* также играет важную роль. Удар киркой выглядит как действие персонажа, а не только как изменение числа в *interface*.

Для правильного отображения инструмента используется привязка кирки к *socket* руки персонажа. Благодаря этому кирка двигается вместе с рукой во время *animation*.

Отдельное значение имеет работа с торговцем. *Unreal Engine 5* позволяет использовать модель персонажа, подключить *animation* и настроить поворот в сторону игрока.

Система *levels* и редактор сцены помогают быстро проверять изменения. Разработчик может поставить торговца на карту, настроить расстояние взаимодействия, проверить *collision* и сразу запустить игру.

Interface проекта также связан с возможностями движка. В игре отображаются ресурсы, золото, зелья, уровень, опыт, *stamina*, подсказки, торговля, прокачка, пауза, настройки и мини-задания.

Особенно важно, что *interface* связан с игровыми данными. Если игрок добыл руду, потратил *stamina* или повысил уровень, данные должны сразу обновиться на экране.

В проекте *RPG FARM* используется игровой цикл добычи, торговли и прокачки. *Unreal Engine 5* подходит для такого цикла, потому что объединяет персонажа, интерактивные объекты, экономику, *interface* и сохранения.

Выбор *Unreal Engine 5* также обоснован возможностью расширения проекта. В будущем можно добавлять новые виды руды, инструменты, задания, карты, эффекты и элементы *interface*.

Для учебной работы важно, что *Unreal Engine 5* позволяет показать не только код, но и результат в виде игровой сцены. Это делает проект понятнее, чем простая консольная программа или набор файлов.

Таким образом, *Unreal Engine 5* выбран потому, что объединяет программирование, визуальную разработку и тестирование игры. Для проекта *C++ RPG FARMER* он подходит как основа трёхмерного игрового проекта с *C++*, *Blueprint*, *assets*, *animation*, *collision* и *interface*. Игровой уровень проекта в редакторе *Unreal Engine 5.7* приведён на рисунке 2.3.

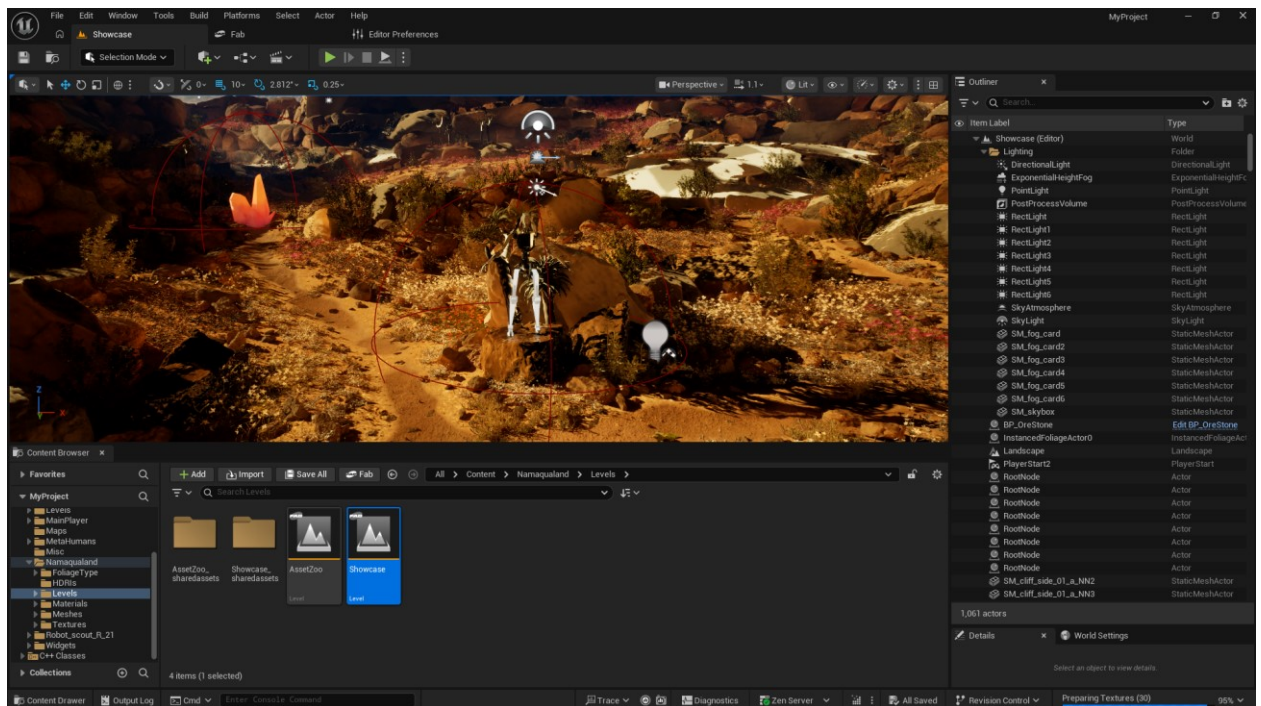


Рисунок 2.3 - Игровой уровень проекта в редакторе *Unreal Engine 5.7*

2.4 Visual Studio 2022

Для написания и проверки программного кода в проекте *C++ RPG FARMER* используется среда разработки *Visual Studio 2022*. Она выбрана

потому, что хорошо подходит для работы с C++ и поддерживается движком *Unreal Engine 5*.

Visual Studio 2022 важна для разработки, потому что в ней удобно редактировать основные классы проекта. Через редактор можно открывать файлы персонажа, *controller*, *interface*, руды, торговца и других игровых систем.

В проекте *RPG FARM* через *Visual Studio 2022* редактируются основные классы игровой логики. Это помогает видеть не один отдельный файл, а общую структуру программы.

Одним из главных преимуществ *Visual Studio 2022* является удобная работа с ошибками компиляции. Если в коде есть ошибка, среда показывает файл, строку и описание проблемы.

Во время разработки часто нужно изменить один класс и проверить, не нарушилась ли другая часть проекта. *Visual Studio 2022* помогает отслеживать такие связи через поиск, переход к объявлению и просмотр мест использования переменных или функций.

Среда разработки удобна для работы с объектно-ориентированной структурой. В проекте используются классы, методы, поля, перечисления и наследование, поэтому логика разделяется по смыслу.

Основные возможности *Visual Studio 2022*, которые используются при разработке проекта, включают:

- редактирование кода на языке C++, благодаря которому создаются классы игровых систем;
- просмотр структуры проекта через *Solution Explorer*, что помогает быстро находить нужные файлы;
- подсказки *IntelliSense*, которые ускоряют написание классов, методов и переменных;
- поиск по проекту, который показывает места использования функций и настроек;

- переход к объявлению и реализации, который помогает быстро перемещаться между файлами;
- окно *Error List*, которое показывает ошибки компиляции;
- окно *Output*, которое отображает сообщения сборки проекта;
- запуск сборки через *Build*, который проверяет готовность проекта к запуску;
- поддержка больших проектов, которая удобна при работе с несколькими игровыми системами.

Для проекта *C++ RPG FARMER* особенно важна возможность быстро находить нужные участки кода. Это упрощает изменение урона кирки, логики *sprint*, системы *stamina* и игровых меню.

Visual Studio 2022 также помогает поддерживать аккуратность кода. Подсказки редактора позволяют заметить ошибки в названиях, типах и подключениях ещё до полной компиляции.

Отдельно важно удобство работы с файлами проекта *Unreal Engine*. Код разделяется на публичные и приватные части, а *Visual Studio 2022* позволяет быстро переключаться между объявлением класса и его реализацией.

В процессе разработки среда используется не только для написания нового кода, но и для исправления существующей логики. После изменения проект собирается и проверяется в редакторе *Unreal Engine 5*.

Такой рабочий процесс позволяет двигаться поэтапно. Сначала определяется задача, затем редактируется код в *Visual Studio 2022*, после этого проект собирается и проверяется в *Unreal Engine 5*.

Важным преимуществом является возможность дальнейшего роста проекта. Если появятся новые типы руды, рабочие заказы, инструменты или меню, их можно будет реализовать через отдельные классы.

Таким образом, *Visual Studio 2022* выбрана как основная среда разработки, потому что она обеспечивает удобное написание, проверку и сопровождение кода на *C++*. Для проекта *C++ RPG FARMER* это делает

разработку более управляемой и технически обоснованной. Работа с исходным кодом проекта в *Visual Studio 2022* приведён на рисунке 2.4.

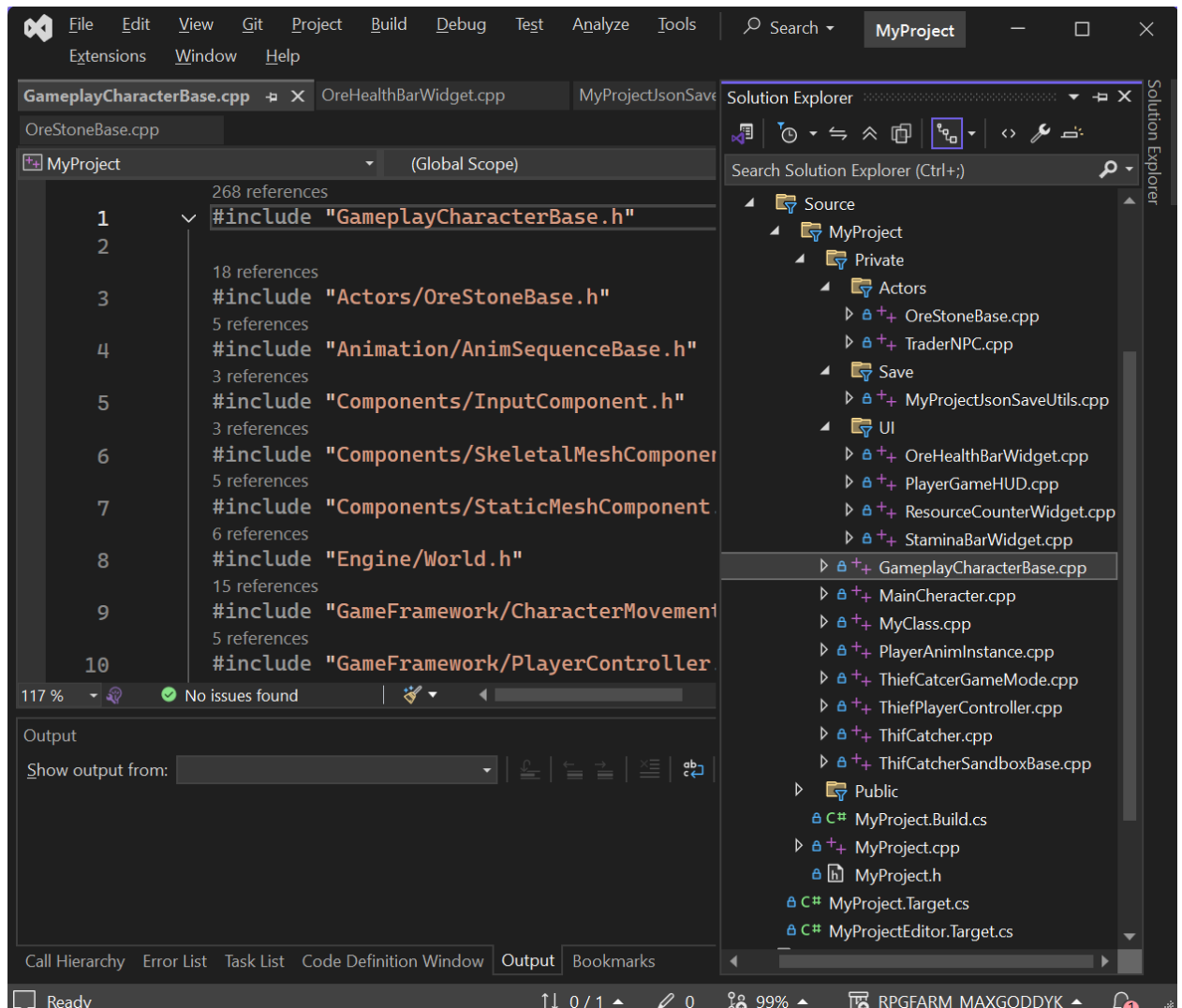


Рисунок 2.4 - Работа с исходным кодом проекта в *Visual Studio 2022*

2.5 Blueprint и assets Unreal Engine

В проекте *C++ RPG FARMER* используются не только классы на *C++*, но и *assets* игрового движка *Unreal Engine*. Такой подход выбран потому, что модели, *animation*, материалы, карты, *sockets* и отдельные параметры объектов удобнее настраивать в редакторе.

Blueprint в *Unreal Engine* представляет собой визуальную систему настройки логики и объектов. В проекте он используется как дополнение к C++, а не как полная замена программированию.

Такой способ разработки удобен для игры, потому что код и визуальная часть должны работать вместе. Логика отвечает за поведение, а *assets* помогают показать персонажа, предметы, окружение и действия на экране.

В проекте есть программная основа персонажа, которая отвечает за игровое поведение. При этом модель, *animation* и параметры скелета подключаются через *assets*.

Одним из важных примеров использования *assets* является кирка. Модель предмета хранится отдельно, а через *socket* она прикрепляется к руке персонажа.

Animation также является важной частью проекта. Для удара киркой используется *animation montage*, который запускает нужное действие во время атаки.

Assets Unreal Engine используются не только для персонажа, но и для окружения. Карта *Showcase* содержит природные объекты, рельеф, камни, растения и другие визуальные элементы.

Также в проекте используется торговец. Его модель, направление, масштаб и положение на карте удобнее проверять через редактор, потому что результат сразу виден на сцене.

Основные виды *assets*, которые применяются в проекте, включают:

- модели персонажа, которые отвечают за внешний вид игрока;
- модель кирки, которая используется как инструмент добычи руды;
- *animation* движения, которая показывает ходьбу, бег и прыжок;
- *animation* удара киркой, которая делает добычу визуально понятной;
- *animation montage*, который запускает удар в нужный момент;
- *sockets* скелета, которые используются для крепления кирки;
- модели окружения, которые формируют игровую карту;

- модель торговца, которая делает взаимодействие понятнее;
- материалы, которые задают внешний вид поверхностей и объектов;
- *levels*, которые разделяют главное меню и игровой процесс.

Использование *Blueprint* и *assets* ускоряет разработку. Положение кирки, модель торговца, параметры *animation* и внешний вид карты проще проверять в редакторе.

При этом визуальная настройка не должна разрушать программную архитектуру. Основная логика остаётся в *C++*, а *Blueprint* и *assets* используются там, где они удобнее кода.

Связка *C++* и *Blueprint* особенно полезна при создании игровых объектов. Базовый класс может быть написан в коде, а визуальные параметры настраиваются через *asset*.

В проекте такой подход применяется к персонажу, торговцу и другим объектам. Код определяет поведение, а *asset* определяет внешний вид и визуальные элементы.

Для учебного проекта это важно с точки зрения демонстрации результата. Когда код связан с *assets*, можно показать не только файлы, но и полноценный игровой процесс.

Использование *assets* помогает улучшать качество игры без полной переработки кода. Можно заменить модель, изменить *animation*, обновить карту или доработать внешний вид *interface*.

Отдельно следует отметить, что *Blueprint* удобен для проверки визуальных событий. При этом сложные расчёты, сохранение данных, прогресс и условия игровых механик остаются в *C++*.

Таким образом, *Blueprint* и *assets Unreal Engine* выбраны как инструменты визуальной и редакторской части проекта. Для *C++ RPG FARMER* это важно, потому что игра должна быть не только технически рабочей, но и визуально понятной. Настройка *Blueprint* и *assets* персонажа в проекте *C++ RPG FARMER* приведён на рисунке 2.5.

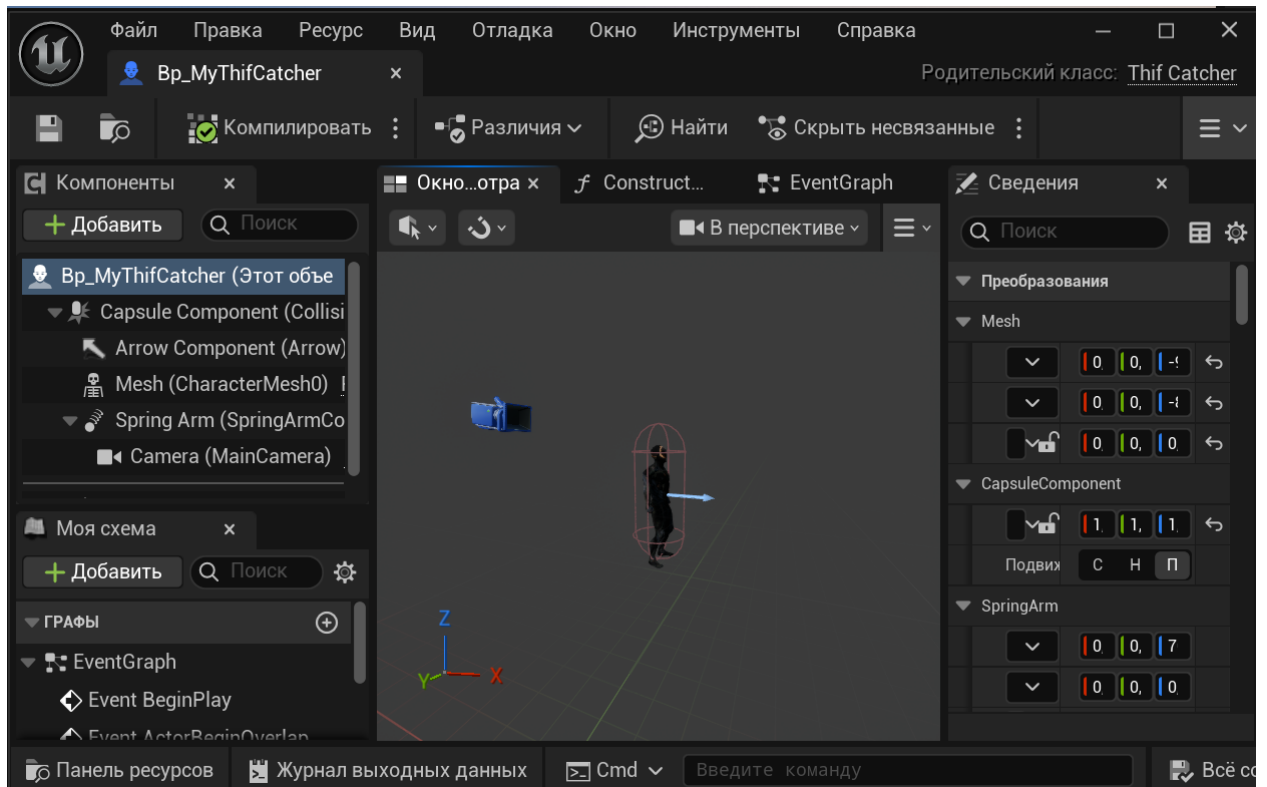


Рисунок 2.5 – Настройка Blueprint и assets персонажа в проекте C++ RPG FARMER

2.6 Unreal Build Tool и система сборки проекта

Для сборки проекта C++ RPG FARMER используется система сборки движка Unreal Engine, в которую входит инструмент Unreal Build Tool. Он отвечает за подготовку, компиляцию и связывание исходного кода.

Обычная программа на C++ может собираться напрямую через компилятор, но проект на Unreal Engine имеет более сложную структуру. В нём есть исходный код, *modules*, заголовочные файлы, макросы движка, *assets*, карты и настройки проекта.

В проекте RPG FAR сборка нужна каждый раз, когда изменяется программная логика. После изменения кода проект должен быть проверен компилятором и только потом запускаться в редакторе.

Unreal Build Tool работает вместе с файлами проекта и средой *Visual Studio 2022*. Разработчик пишет код в *Visual Studio 2022*, запускает *build*, а система *Unreal Engine* определяет, какие файлы нужно скомпилировать.

Важной частью проекта являются *modules*. *Module* в *Unreal Engine* можно представить как отдельную программную часть, в которой собраны классы и зависимости.

Система сборки выполняет несколько важных задач:

- проверяет синтаксис кода на языке *C++*;
- обрабатывает специальные макросы *Unreal Engine*;
- подключает необходимые *modules* движка;
- определяет зависимости между файлами проекта;
- формирует рабочие файлы для запуска в редакторе;
- помогает поддерживать порядок в структуре проекта.

Для учебного проекта система сборки важна как способ контроля качества. Успешная сборка показывает, что классы связаны правильно, методы объявлены и реализованы корректно, а зависимости подключены.

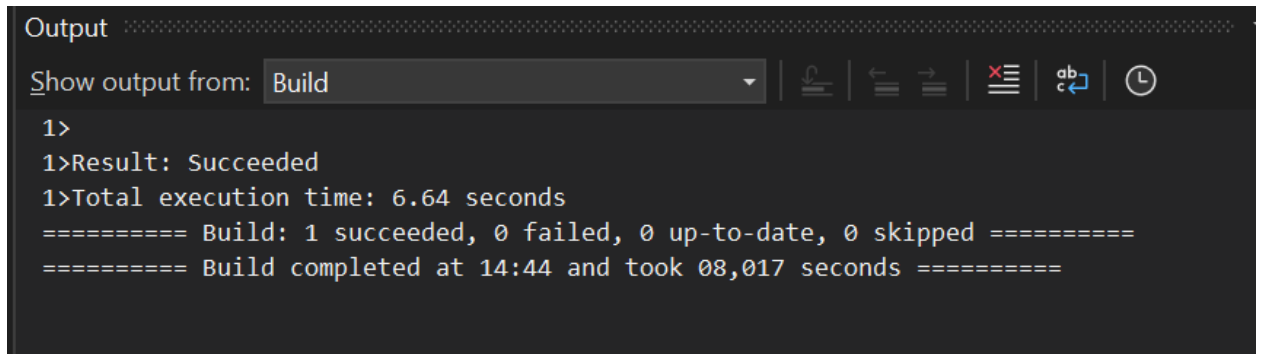
Ошибки сборки помогают находить проблемы раньше, чем они проявятся в игре. Компилятор сообщает о неправильных типах данных, отсутствующих подключениях, ошибках в названиях и нарушенной структуре функций.

В проекте *C++ RPG FARMER* сборка связана с реальными игровыми задачами. После изменения классов персонажа, *controller*, *HUD*, руды или торговца проект должен быть собран перед проверкой в редакторе.

Система сборки также помогает поддерживать масштабируемость проекта. Если будут добавлены новые классы, виды руды, улучшения или *subsystem*, они должны корректно подключаться к *module* проекта.

Отдельно важно, что *Unreal Build Tool* работает с системой отражения движка. Благодаря этому классы, свойства и функции из кода могут быть доступны в редакторе, *assets* и *Blueprint*.

Таким образом, *Unreal Build Tool* и система сборки выбраны как обязательная часть разработки на *Unreal Engine 5*. Они обеспечивают проверку кода, подключение *modules*, обработку зависимостей и подготовку проекта к запуску. Сборка проекта *C++ RPG FARMER* с использованием *Unreal Build Tool* приведён на рисунке 2.6.



```

Output
Show output from: Build
1>
1>Result: Succeeded
1>Total execution time: 6.64 seconds
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 14:44 and took 08,017 seconds =====
  
```

Рисунок 2.6 - Сборка проекта *C++ RPG FARMER* с использованием *Unreal Build Tool*

2.7 *Git* и контроль версий

Для контроля изменений в проекте *C++ RPG FARMER* используется система *Git*. Она нужна для хранения истории разработки, сравнения файлов и возврата к рабочему состоянию при ошибках. Для игрового проекта это важно, потому что одновременно меняются код, настройки, *assets*, *levels*, *interface* и файлы сохранений.

Во время разработки постоянно появляются новые правки и улучшения. Каждое изменение может затрагивать несколько файлов, поэтому без контроля версий сложнее понять, когда появилась ошибка и какие действия к ней привели.

Git позволяет сохранять состояние проекта через *commits*. Такой *commit* можно считать контрольной точкой разработки. Если после изменений программа работает неправильно, текущее состояние можно сравнить с прошлым и быстрее найти причину ошибки.

Для проекта на *Unreal Engine 5* контроль версий полезен особенно сильно, потому что игра состоит не только из кода. Текстовые файлы, например *C++*, *INI* и *JSON*, удобно сравнивать, а бинарные *assets* и *maps* важно хранить в истории, чтобы не потерять результат настройки.

Главная польза *Git* заключается в том, что разработка становится более управляемой. Разные задачи можно фиксировать отдельно, поэтому исправление ошибки, изменение баланса и настройка *interface* не смешиваются между собой.

Основные причины использования *Git* в проекте заключаются в следующем:

- хранение истории изменений позволяет видеть развитие проекта от простой версии к более полной системе;
- сравнение файлов помогает находить изменённые строки при исправлении ошибки или добавлении функции;
- контроль состояния проекта снижает риск потери рабочей версии после неудачной правки;
- разделение задач по *commits* помогает отличать исправление ошибки от добавления новой возможности;
- работа с *branches* позволяет проверять новые идеи без изменения основной версии;
- хранение текстовых файлов помогает отслеживать изменения в коде, настройках, сохранениях и конфигурации;
- возможность отката помогает безопаснее экспериментировать с *interface*, балансом и логикой игры.

В учебном проекте *Git* также полезен как подтверждение поэтапной разработки. По истории изменений можно показать, что работа выполнялась постепенно: сначала создавалась основа, затем исправлялись ошибки, после этого добавлялись новые элементы и улучшалось оформление.

Контроль версий помогает при работе с ошибками. Если после очередной правки поведение проекта изменилось, можно посмотреть последние изменённые файлы и быстрее определить источник проблемы. *Git* также снижает риск при крупных изменениях. Перед экспериментом можно сохранить рабочее состояние, а если новая идея окажется неудачной, вернуться к предыдущему варианту. Для проекта *C++ RPG FARMER* важны не только *commits*, но и понятные сообщения к ним. Хорошее сообщение кратко объясняет, что было сделано, поэтому по истории легче понять ход разработки. При работе с *Unreal Engine* нужно учитывать разницу между текстовыми и бинарными файлами. Код и настройки удобно сравнивать, а *assets* и *maps* обычно нельзя полноценно прочитать как текст, но они всё равно должны сохраняться в истории проекта. Выбранные технологии разработки приведён на таблице 2.1.

Использование *Git* не заменяет тестирование, а дополняет его. Даже если проект собирается и запускается, история изменений помогает понять, как он пришёл к текущему состоянию. Таким образом, *Git* выбран для проекта *C++ RPG FARMER* как инструмент безопасной и понятной разработки. Он позволяет хранить историю, сравнивать изменения, фиксировать рабочие состояния, разделять задачи и снижать риск потери проекта. История изменений проекта в системе контроля версий *Git* приведён на рисунке 2.7.

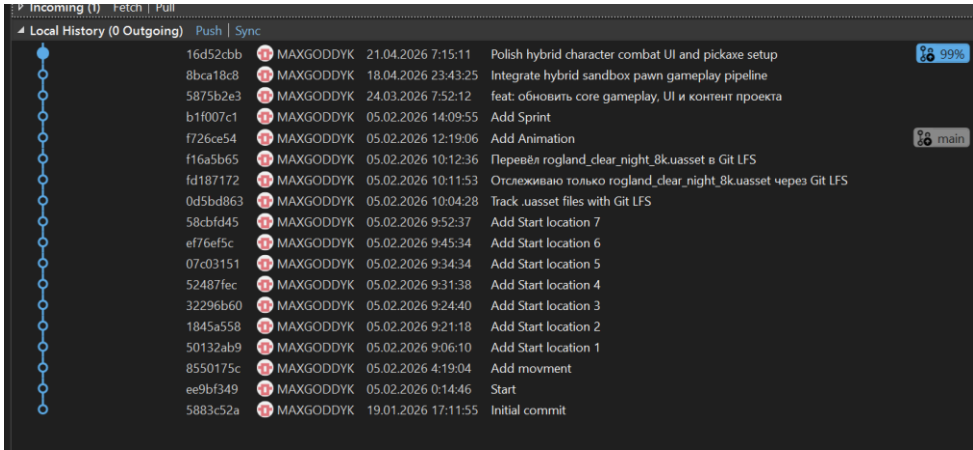


Рисунок 2.7 - История изменений проекта в системе контроля версий *Git*

Таблица 2.1 – Выбранные технологии разработки

Название технологии	Версия	Назначение
Unreal Engine 5	5.7	Игровой движок: рендеринг графики, физика, сборка проекта
C++	C++20	Разработка игровой логики, персонажа, NPC и игровых систем
Blueprints	UE 5.7	Визуальное скриптование и прототипирование геймплея
Enhanced Input	UE 5.7	Обработка ввода игрока (клавиатура и мышь)
UMG (Unreal Motion Graphics)	UE 5.7	Создание интерфейса: HUD, полосы здоровья и стамины
JSON (Json / JsonUtilities)	UE 5.7	Сохранение и загрузка прогресса игры
Visual Studio	2022	Среда разработки и компиляции кода C++

3. ПРОЦЕСС ПРОЕКТИРОВАНИЯ В СРЕДЕ РАЗРАБОТКИ

3.1 Описание порядка использования *Visual Studio* и *Unreal Engine 5*

Visual Studio используется как основная среда для работы с кодом на C++. В ней удобно открывать файлы, искать методы, проверять ошибки компиляции и запускать сборку. Для проектов на *Unreal Engine* это стандартный вариант, потому что движок создаёт решение и подключает нужные модули.

Во время разработки *Visual Studio* помогает контролировать структуру исходного кода и быстро находить технические ошибки. Компилятор показывает проблемы с типами, объявлениями функций, подключением файлов и зависимостями между частями программы.

Редактор *Unreal Engine 5* используется для настройки визуальной части проекта. Через него можно открывать *levels*, размещать объекты, подключать *assets*, проверять материалы, настраивать *Blueprints* и запускать проект в режиме *Play*.

Редактор особенно важен там, где требуется визуальная проверка. Он позволяет оценить расположение объектов на сцене, масштаб, поворот, внешний вид моделей, освещение и зоны взаимодействия без ручного изменения всех параметров в коде.

Также *Unreal Engine* удобен для проверки *interface*. После запуска проекта разработчик сразу видит, как отображаются игровые панели, меню и другие элементы на экране. Это позволяет быстрее оценить общее расположение элементов, их размер, читаемость и удобство восприятия во время игры. Благодаря такой наглядности ошибки оформления, смещения, наложения или неудачные размеры отдельных блоков можно находить и исправлять значительно быстрее. Это особенно важно при разработке

игрового проекта, где *interface* должен быть не только красивым, но и понятным для пользователя.

3.2 Структура проекта в *Visual Studio*

Структура проекта помогает разделить исходный код, ресурсы, настройки и рабочие данные. Такое разделение делает проект понятнее, упрощает поиск нужных файлов и снижает риск случайного изменения не той части проекта.

Для проекта на *Unreal Engine 5* важно отделять папку с кодом от папки с игровыми ресурсами. Код обычно хранится в *Source*, а визуальные материалы, *levels*, *Blueprints* и другие *assets* находятся в *Content*.

Отдельно выделяются рабочие папки, которые создаются во время запуска редактора или игры. Они не являются основной частью программной логики, но помогают хранить временные данные, журналы, сохранения и результаты работы проекта.

Поддержание структуры важно для дальнейшего развития. Если файлы расположены хаотично, проект становится трудно сопровождать, поэтому разделение по назначению делает работу более стабильной и понятной.

3.3 Работа с исходным кодом проекта

Исходный код в проекте на *Unreal Engine 5* отвечает за основную программную часть. Через него задаются правила поведения объектов, обработка ввода, работа игровых состояний, связь с *interface* и обмен данными между системами.

В проектах на *C++* обычно используются заголовочные файлы и файлы реализации. В заголовочных файлах описываются классы, свойства и методы,

а в файлах реализации размещается логика их работы. Такое разделение помогает быстрее понимать структуру программы.

Заголовочный файл удобен для общего просмотра возможностей класса. Файл реализации нужен тогда, когда требуется изучить конкретный алгоритм, проверку, вычисление или обработку действия пользователя.

Основные папки исходного кода выполняют следующие задачи:

- папка *Public* хранит объявления классов, методов, свойств и структур;
- папка *Private* хранит реализацию логики и поведение методов;
- папка *Actors* содержит классы игровых объектов;
- папка *UI* содержит классы, связанные с отображением *interface*;
- папка *Save* содержит код для сохранения и загрузки данных.

Работа с кодом выполняется через *Visual Studio*. В ней удобно просматривать дерево файлов, переходить к определениям, искать функции, исправлять ошибки компиляции и пересобирать проект после изменения исходных файлов.

Связь *C++* и *Unreal Engine 5* позволяет совмещать строгую программную логику с визуальной настройкой в редакторе. Такой подход особенно удобен для учебного проекта, потому что показывает не только работу с исходным кодом, но и даёт возможность сразу наблюдать, как этот код влияет на поведение объектов, интерфейс и общий игровой процесс. Разработчик может реализовать нужную механику на уровне программы, а затем быстро проверить результат в игровой среде, увидеть возможные ошибки и при необходимости сразу внести исправления. Это делает процесс разработки более наглядным, понятным и удобным, а также позволяет лучше продемонстрировать связь между программной частью проекта и её визуальным отображением в готовой игровой сцене. Кроме того, такой способ работы помогает быстрее сравнивать ожидаемый результат с фактическим поведением игры, что особенно важно при поэтапной разработке и тестировании отдельных функций. В результате проект становится не только

технически обоснованным, но и более удобным для анализа, демонстрации и дальнейшего совершенствования.

3.4 Работа с ресурсами и *assets*

Работа с ресурсами выполняется через папку *Content*. В ней хранятся *levels*, *Blueprint assets*, материалы, модели, текстуры и другие элементы, которые используются редактором *Unreal Engine 5*.

Папка *Content* нужна для связи программной части с визуальной частью проекта. Код задаёт правила работы, а ресурсы позволяют разместить объекты на сцене, настроить внешний вид и проверить результат в редакторе.

Основные элементы, с которыми выполняется работа в папке *Content*:

- *levels* для запуска и проверки проекта;
- *Blueprint assets* для настройки объектов в редакторе;
- *materials*, *textures* и модели для визуального оформления;
- элементы, необходимые для проверки *HUD*, взаимодействия и игровых состояний.

Blueprint assets удобны тем, что позволяют использовать основу, написанную на C++, в визуальной форме. Это ускоряет настройку объектов, потому что часть параметров можно менять через редактор без полного изменения кода.

Визуальные материалы также важны для восприятия проекта. *Materials* и *textures* помогают различать объекты, задают внешний вид сцены и делают игровое пространство более понятным для пользователя.

Работа с ресурсами требует постоянной проверки в редакторе. После изменения *level* или *asset* проект можно запустить в режиме *PIE*, чтобы быстро увидеть результат без отдельной финальной сборки.

Таким образом, папка *Content* является рабочей частью проекта. Через неё выполняется настройка *levels*, размещение объектов, проверка *Blueprint assets*, подбор материалов и визуальная проверка результата.

3.5 Конфигурационные файлы проекта

Конфигурационные файлы находятся в папке *Config*. Они задают параметры, которые применяются при запуске проекта и работе редактора *Unreal Engine 5*.

В отличие от кода, такие файлы не описывают игровую логику напрямую, но определяют условия её запуска.

Основное значение имеют файлы *DefaultEngine.ini*, *DefaultInput.ini* и *DefaultGame.ini*. Они используют формат *INI*, где параметры разделяются по блокам и могут быть прочитаны обычным текстовым редактором.

Файл *DefaultEngine.ini* отвечает за настройки движка, стартовые карты, игровой режим и часть параметров отображения. Благодаря этому проект запускается с нужной конфигурацией, а не со стандартными настройками пустого шаблона.

Основные задачи файла *DefaultEngine.ini* заключаются в следующем:

- выбор карты, которая открывается при запуске игры;
- выбор карты, которая открывается при запуске редактора;
- назначение основного игрового режима проекта;
- хранение параметров графики и отображения;
- подключение общих настроек, необходимых для работы проекта.

Файл *DefaultInput.ini* отвечает за управление. В нём задаются действия игрока и клавиши, которые с ними связаны. Такая настройка важна, потому что без корректного *input* пользователь не сможет нормально управлять проектом.

В настройках управления обычно используются действия и оси. Действия выполняются при нажатии клавиши, а оси отвечают за направление движения или поворот камеры. Такой подход удобен для трёхмерных проектов, где важны и команды, и непрерывное движение.

К основным действиям управления относятся:

- прыжок персонажа через клавишу *Space Bar*;
- включение *sprint* через клавишу *Left Shift*;
- атака через кнопку *Left Mouse Button*;
- взаимодействие через клавишу *E*;
- открытие торгового меню через клавишу *B*;
- открытие меню прогресса через клавишу *P*;
- выход назад или пауза через клавишу *Escape*.

Файл *DefaultGame.ini* хранит общие параметры проекта. Обычно он меньше по объёму, чем *DefaultEngine.ini* и *DefaultInput.ini*, но также входит в стандартную структуру проекта на *Unreal Engine 5*.

Конфигурационные файлы помогают отделить настройки проекта от исходного кода. Это удобно, потому что карту запуска, управление и часть параметров окружения можно проверить без переписывания логики на *C++*.

Проверка конфигурационных файлов выполняется по нескольким направлениям:

- открывается ли правильная карта при запуске проекта;
- используется ли нужный игровой режим;
- работает ли управление с клавиатуры и мыши;
- открываются ли меню через назначенные клавиши;
- сохраняется ли связь между настройками проекта и кодом на *C++*.

Таким образом, папка *Config* является важной частью проекта. Она не содержит основную игровую механику, но определяет стартовую карту, игровой режим, управление и общие параметры запуска

3.6 Средства проверки и отладки

Проверка проекта выполняется на нескольких этапах разработки. После изменения кода проект собирается, затем запускается в редакторе, а результат проверяется в режиме *PIE*. Такой порядок помогает находить ошибки до финальной сборки. Основным средством проверки является *Unreal Engine Editor*. В нём можно открыть *level*, запустить игровой режим, проверить расположение объектов, работу камеры, отображение *HUD* и общее состояние сцены. На рисунке 3.1 показан запуск проекта в режиме проверки через редактор *Unreal Engine 5*.

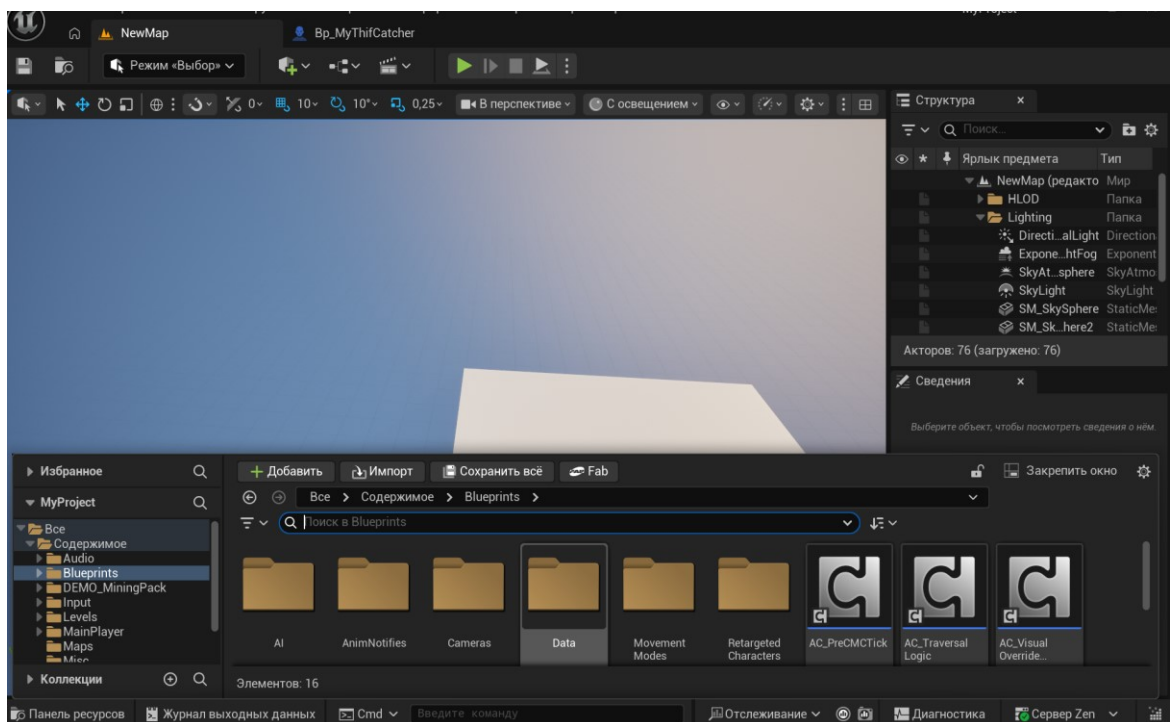


Рисунок 3.1 - Проверка основной карты проекта в редакторе *Unreal Engine 5*

Режим *PIE* используется для запуска проекта прямо внутри редактора. Это удобно, потому что разработчику не нужно каждый раз создавать отдельную сборку. При обнаружении ошибки можно остановить запуск, внести правку и сразу повторить проверку. В первую очередь проверяется корректность запуска проекта и работа базового ввода. Нужно убедиться, что

клавиатура и мышь распознаются правильно, а настройки *input* не конфликтуют с общей структурой проекта.

Основные элементы, которые проверяются во время запуска:

- открытие нужной карты в редакторе;
- корректный запуск режима *PIE*;
- реакция проекта на клавиатуру и мышь;
- отсутствие ошибок компиляции после правок;
- отсутствие критических ошибок в окне вывода *Unreal Engine*;
- корректное завершение тестового запуска.

Дополнительно проверяется отображение основных экранных элементов. На этом этапе важно убедиться, что *interface* появляется без ошибок, не перекрывает рабочую область и не мешает проверке проекта в редакторе.

После проверки запуска выполняется повторная сборка проекта в *Visual Studio*. Если в коде есть ошибка, среда разработки показывает сообщение, файл и строку, где возникла проблема. Это помогает быстрее исправлять неточности в объявлениях, подключениях и вызовах функций.

Проверка проекта включает следующие этапы:

- сборка проекта после изменения исходного кода;
- запуск проекта в режиме *PIE*;
- проверка открытия нужной карты;
- проверка реакции на базовый ввод;
- просмотр сообщений в окне вывода редактора;
- повторный запуск после внесения исправлений.

3.7 Рабочие файлы и сохранения

Во время разработки используются не только исходный код, ресурсы и конфигурационные файлы, но и рабочие данные, которые появляются после

запуска проекта. Такие файлы помогают проверить, сохраняются ли изменяемые параметры и корректно ли проект восстанавливает состояние после повторного открытия. Для хранения изменяемых данных удобно использовать формат *JSON*. Его можно открыть обычным текстовым редактором, быстро просмотреть структуру и проверить значения без специальных программ. Это делает формат полезным для отладки и учебной разработки. Рабочие файлы сохранений обычно размещаются отдельно от исходного кода. Такой подход помогает не смешивать программную часть проекта с данными, которые создаются во время запуска и проверки.

К рабочим данным проекта могут относиться:

- пользовательские настройки;
- параметры управления;
- значения громкости;
- выбранный язык *interface*;
- данные прогресса;
- временные результаты проверки.

Файл *Settings.json* используется для хранения настроек пользователя. В нём могут находиться параметры громкости, масштаба меню, чувствительности мыши, выбранного языка и назначенных клавиш управления.

К данным, которые могут сохраняться в *Settings.json*, относятся:

- общая громкость;
- громкость музыки и звуковых эффектов;
- масштаб элементов меню;
- чувствительность мыши;
- выбранный язык *interface*;
- назначенные клавиши управления.

Файл *Character.json* используется для хранения данных, связанных с состоянием персонажа. Такой файл помогает проверить, меняются ли

значения после действий пользователя и загружаются ли они при следующем запуске.

К данным, которые могут сохраняться в *Character.json*, относятся:

- текущее и максимальное значение *stamina*;
- количество ресурсов;
- количество золота;
- количество предметов восстановления *stamina*;
- уровень и опыт;
- очки улучшений;
- скорость движения и скорость *sprint*.

Использование *JSON* удобно для проверки, потому что данные имеют понятную структуру. Если значение в игре отображается неправильно, можно открыть файл сохранения и сравнить записанные данные с тем, что видно во время запуска проекта.

При проверке сохранений выполняются следующие действия:

- запуск проекта;
- изменение пользовательских данных;
- закрытие проекта;
- открытие файлов *JSON*;
- сравнение записанных значений;
- повторный запуск и проверка загрузки данных.

Важно учитывать, что папка *Saved* относится к рабочим данным. В ней могут находиться временные файлы, журналы и результаты запусков, поэтому её нужно отличать от папок с исходным кодом, ресурсами и конфигурацией.

Таким образом, рабочие файлы и сохранения помогают проверить, что проект не только запускается в среде разработки, но и корректно хранит изменяемые данные. Для учебного проекта это важно, потому что сохранения показывают связь между запуском программы, действиями пользователя и записанными значениями.

4. ЭТАПЫ РАЗРАБОТКИ

4.1 Создание проекта приложения

На первом этапе был подготовлен проект *MyProject* на базе *Unreal Engine 5.7*. В качестве основы использовался проект с поддержкой *C++*, так как основная логика реализуется в отдельных классах исходного кода, а не только в настройках редактора. Такой подход подходит для учебной работы, так как позволяет показать работу с классами, объектами, управлением, *HUD*, сохранениями и игровыми системами. После создания проекта была сформирована стандартная структура папок *Unreal Engine*. В корневом каталоге находятся *Source*, *Content*, *Config*, *Saved*, *Binaries* и *Intermediate*. Папка *Source* содержит код на *C++*, *Content* хранит *levels*, *Blueprint assets*, модели, звуки, *materials* и элементы *interface*, *Config* содержит настройки проекта, а *Saved* используется для рабочих данных и сохранений в формате *JSON*. Корневая структура проекта *MyProject* приведён на рисунке 4.1.

Имя	Дата изменения	Тип	Размер
.github	26.04.2026 18:34	Папка с файлами	
Binaries	22.04.2026 8:06	Папка с файлами	
Build	21.04.2026 12:22	Папка с файлами	
Config	28.04.2026 7:59	Папка с файлами	
Content	27.04.2026 4:04	Папка с файлами	
DerivedDataCache	22.04.2026 8:07	Папка с файлами	
Intermediate	18.05.2026 8:43	Папка с файлами	
Saved	18.05.2026 8:03	Папка с файлами	
Source	04.02.2026 21:18	Папка с файлами	
Tools	11.05.2026 8:12	Папка с файлами	
.vsconfig	04.02.2026 21:18	Файл "VSCONFIG"	1 КБ
MyProject.sln	19.03.2026 5:58	Visual Studio Solu...	173 КБ
MyProject	18.04.2026 1:35	Unreal Engine Proj...	2 КБ
README	17.04.2026 13:59	Исходный файл ...	21 КБ
RPG_FARM_Official_TZ_Reworked	27.04.2026 3:18	Документ Micros...	62 КБ

Рисунок 4.1 - Корневая структура проекта *MyProject*

Основные папки проекта имеют следующее назначение:

- папка *Source* содержит исходный код проекта на языке C++;
- папка *Content* используется для *levels*, визуальных ресурсов, *Blueprint assets* и игровых объектов;
- папка *Config* хранит настройки запуска, управления и параметров движка;
- папка *Saved* содержит рабочие данные редактора, журналы и файлы сохранений;
- папки *Binaries* и *Intermediate* используются движком для сборки и временных файлов.

Важным файлом проекта является *MyProject.uproject*. В нём указаны версия движка, основной модуль и список подключённых *plugins*. В поле *EngineAssociation* указана версия 5.7, а модуль *MyProject* имеет тип *Runtime*, поэтому используется во время запуска игры. В файле *MyProject.uproject* также подключены дополнительные *plugins*, включая *AnimationWarping*, *MotionWarping*, *ControlRig*, *IKRig*, *GameplayCameras* и другие компоненты. Они расширяют возможности проекта, но основная логика игры остаётся в собственных C++ классах. Файл проекта *MyProject.uproject* с настройками движка и модуля приведён на рисунке 4.2.

```
{
    "FileVersion": 3,
    "EngineAssociation": "5.7",
    "Category": "",
    "Description": "",
    "Modules": [
        {
            "Name": "MyProject",
            "Type": "Runtime",
            "LoadingPhase": "Default"
        }
    ]
}
```

Рисунок 4.2 - Файл проекта *MyProject.uproject* с настройками движка и модуля

После подготовки файла проекта была настроена структура исходного кода. В папке *Source/MyProject* находятся файлы основного модуля, файл

сборки *MyProject.Build.cs*, а также папки *Public* и *Private*. Такое разделение является стандартным для проектов на *Unreal Engine*.

Папка *Public* содержит заголовочные файлы, где описываются классы, переменные, функции и структуры. Папка *Private* содержит файлы реализации, где находится логика работы методов. Благодаря этому сначала можно увидеть возможности класса, а затем открыть реализацию и изучить его поведение.

В проекте отдельные классы выделены для основных игровых систем. Персонаж, *controller*, *GameMode*, *HUD*, руда, торговец и система *JSON*-сохранений расположены в разных файлах. Такое разделение помогает не смешивать всю логику в одном месте и упрощает дальнейшую поддержку проекта.

Файл *MyProject.Build.cs* отвечает за подключение модулей для сборки проекта. В нём указаны зависимости *Core*, *CoreUObject*, *Engine*, *InputCore*, *Enhanced Input*, *UMG*, *AnimGraphRuntime*, *Slate*, *SlateCore*, *Json*, *JsonUtilities*, а также модули для анимации, камеры, задач и игровых систем.

Подключённые модули используются в разных частях проекта:

- модули *Core* и *CoreUObject* обеспечивают базовые типы, объекты и системные возможности движка;
- модуль *Engine* предоставляет игровые классы, компоненты, актёров и работу с игровым миром;
- модули *InputCore* и *EnhancedInput* используются для обработки управления;
- модули *UMG*, *Slate* и *SlateCore* применяются для пользовательского *interface* и экранных элементов;
- модули *Json* и *JsonUtilities* используются для сохранения настроек и данных персонажа.

Подготовка структуры также включала настройку папки *Content*. В ней находятся игровые *levels*, *Blueprint assets*, папки с персонажем, ресурсами,

звуками и элементами *interface*. Основные элементы: *NewMap*, *Showcase*, *Blueprints*, *MainPlayer*, *Audio* и *Namaqualand*.

Отдельное значение имеет разделение карт проекта. *NewMap* используется как стартовое лобби, а *Showcase* — для основного игрового процесса. Это отделяет начальное состояние игры от активной игровой сцены.

На этапе подготовки были определены основные классы проекта. *ThiefCatcerGameMode* отвечает за игровой режим, *ThiefPlayerController* управляет вводом и переходами, *GameplayCharacterBase* содержит логику персонажа, *PlayerGameHUD* отвечает за отображение *interface*, а *OreStoneBase* и *TraderNPC* описывают игровые объекты.

Изменение поведения руды выполняется через *OreStoneBase*. Изменение торгового меню выполняется через *PlayerGameHUD* и логику *controller*. Изменение параметров персонажа выполняется через *GameplayCharacterBase*.

На этапе подготовки были выполнены следующие действия:

- создан проект *Unreal Engine 5.7* с поддержкой *C++*;
- проверен файл *MyProject.uproject* и основной модуль *MyProject*;
- подготовлены папки *Source*, *Content*, *Config* и *Saved*;
- настроена структура исходного кода с папками *Public* и *Private*;
- выделены основные классы для персонажа, *controller*, *GameMode*, *HUD*, руды, торговца и сохранений.

Таким образом, на первом этапе была подготовлена структура проекта. После этого стало возможным добавить стартовое лобби, переход на карту *Showcase*, управление персонажем, систему *stamina*, добычу руды, торговлю, прокачку, *HUD* и сохранение данных в формате *JSON*.

4.2 Создание персонажа

После подготовки структуры проекта была реализована система стартового *lobby*. Она позволяет пользователю сначала увидеть стартовый

экран, открыть настройки, проверить управление и только после этого начать игровую сессию. Для этого используется карта *NewMap*, которая открывается при запуске проекта. Игровая карта отделена от стартового *lobby*. Основной игровой процесс происходит на карте *Showcase*, расположенной по пути *Namaqualand/Levels*. На ней проверяются движение персонажа, добыча руды, взаимодействие с торговцем, торговля, прокачка, *HUD* и сохранение прогресса.

На рисунке 4.3 показана схема перехода от стартового *lobby* к игровой карте.

```
MenuMapName = TEXT("NewMap");
GameplayMapName = TEXT("/Game>Namaqualand/Levels/Showcase");
LoadingScreenDelay = 0.15f;

if (UWorkOrderSubsystem* WorkOrderSubsystem = GetWorkOrderSubsystem())
{
    WorkOrderSubsystem->PrepareSelectedOrderForLaunch();
}

GetWorldTimerManager().SetTimer(
    LoadingMapTimerHandle,
    this,
    &AThiefPlayerController::OpenGameplayMap,
    LoadingScreenDelay,
    false);

void AThiefPlayerController::OpenGameplayMap()
{
    if (!IsInMenuMap())
    {
        return;
    }

    UGameplayStatics::OpenLevel(this, GameplayMapName);
}
```

Рисунок 4.3 - Переход от стартового *lobby* к игровой карте *Showcase*

За запуск стартового состояния отвечает несколько частей проекта. Основная карта задаётся в файле *DefaultEngine.ini*, а переход между картами выполняется в классе *AThiefPlayerController*. Этот класс хранит имена карт,

обрабатывает действия из *HUD*, показывает экран загрузки и открывает игровую карту. В конструкторе *AThiefPlayerController* задаются начальные значения. Переменная *MenuMapName* хранит имя *NewMap*, переменная *GameplayMapName* содержит путь */Game/Namaqualand/Levels/Showcase*, а *LoadingScreenDelay* задаёт задержку перед открытием игровой карты.

Основные переменные, связанные с переходом между картами:

- *MenuMapName* хранит имя стартовой карты *NewMap*;
- *GameplayMapName* хранит путь к игровой карте *Showcase*;
- *LoadingScreenDelay* задаёт задержку перед открытием игровой карты;
- *bLoadingGameplayMap* показывает состояние загрузки;
- *LobbyMusic* хранит звук стартового экрана.

Функция *BeginPlay* сначала вызывает базовую реализацию родительского класса, затем при необходимости подключает *APlayerGameHUD* через *ClientSetHUD*. После этого выполняется загрузка настроек и подготовка стартового состояния проекта. Инициализация *controller* при запуске проекта приведён на рисунке 4.4.

```
void AThiefPlayerController::BeginPlay()
{
    Super::BeginPlay();

    if (!GetHUD() || !GetHUD()->IsA(APlayerGameHUD::StaticClass()))
    {
        ClientSetHUD(APlayerGameHUD::StaticClass());
    }

    LoadSettingsFromJson();
    bLoadingGameplayMap = false;
    ApplyMenuInputState();
    ApplyLookSettings();
    ApplyAudioSettings();

    GetWorldTimerManager().ClearTimer(AudioRefreshTimerHandle);
    GetWorldTimerManager().SetTimer(
        AudioRefreshTimerHandle,
        this,
        &AThiefPlayerController::ApplyWorldAudioVolumes,
        1.0f,
        true,
        0.25f);
}
```

Рисунок 4.4 - Инициализация *controller* при запуске проекта

В классе *AThiefCatcerGameMode* переопределена функция *SpawnDefaultPawnFor_Implementation*. Она проверяет имя текущей карты и не создаёт *Pawn* на карте *NewMap*. Это нужно потому, что в стартовом *lobby* пользователь работает только с меню и не должен управлять персонажем.

На рисунке 4.5 можно показать реализацию функции *SpawnDefaultPawnFor_Implementation*.

```
APawn* AThiefCatcerGameMode::SpawnDefaultPawnFor_Implementation(AController*
NewPlayer, AActor* StartSpot)
{
    if (GetWorld())
    {
        FString MapName = GetWorld()->GetMapName();
        MapName.RemoveFromStart(GetWorld()->StreamingLevelsPrefix);
        if (FPackageName::GetShortName(MapName).Equals(TEXT("NewMap"),
ESearchCase::IgnoreCase))
        {
            return nullptr;
        }
    }

    return Super::SpawnDefaultPawnFor_Implementation(NewPlayer, StartSpot);
}
```

Рисунок 4.5 - Отключение создания *Pawn* на стартовой карте *NewMap*

Стартовый экран отображается в классе *APlayerGameHUD*. Он рисуется через *Canvas* и содержит основные панели, название проекта, описание игрового цикла и кнопки управления. Среди кнопок есть запуск игры, открытие настроек и выход из проекта.

Стартовый экран выполняет не только декоративную, но и практическую функцию. Пользователь может подготовиться к запуску, открыть настройки, проверить управление и перейти на игровую карту.

На стартовом *lobby* пользователь может выполнить следующие действия:

- начать игру и перейти на карту *Showcase*;
- открыть настройки звука, управления и *interface*;
- проверить текущие клавиши управления;
- выйти из проекта через кнопку завершения;

- увидеть краткую подсказку о цикле игры.

Кнопка запуска связана с действием *StartGame*. Когда пользователь выбирает её, *HUD* передаёт команду в *AThiefPlayerController*, после чего вызывается функция *StartGameplayFromMenu*.

Функция *StartGameplayFromMenu* проверяет, что пользователь находится на карте *NewMap* и переход ещё не начат. Затем она закрывает активные меню, включает состояние загрузки, применяет режим ввода и подготавливает переход на карту *Showcase*.

Если *LoadingScreenDelay* больше нуля, переход выполняется не сразу. Для этого используется *timer*, который через короткую задержку вызывает функцию *OpenGameplayMap*. В проекте задержка составляет 0.15 секунды.

Этапы запуска игровой карты выглядят следующим образом:

- пользователь нажимает кнопку запуска игры в стартовом *lobby*;
- *HUD* передаёт действие *StartGame* в *controller*;
- функция *StartGameplayFromMenu* включает состояние загрузки;
- *APlayerGameHUD* отображает *loading screen*;
- после задержки вызывается функция *OpenGameplayMap*;
- функция *OpenGameplayMap* открывает карту *Showcase*.

На рисунке 4.6 показан фрагмент функции *StartGameplayFromMenu*, которая запускает переход на игровую карту.

Промежуточный экран загрузки рисуется функцией *DrawLoadingScreen* в классе *APlayerGameHUD*. Он появляется, когда *controller* находится в состоянии *bLoadingGameplayMap*. На экране отображаются затемнение, центральная панель, текст загрузки и простая анимированная полоса.

Функция *DrawLoadingScreen* использует время игрового мира для изменения текста и движения полосы. Благодаря этому пользователь видит, что проект выполняет переход между картами, а не завис.

Открытие игровой карты выполняется функцией *OpenGameplayMap*. Она вызывает стандартную функцию *OpenLevel* и передаёт ей имя карты

Showcase. После загрузки новой карты персонаж создаётся обычным образом, потому что блокировка *Pawn* действует только для *NewMap*.

```
void AThiefPlayerController::StartGameplayFromMenu()
{
    if (!IsInMenuMap() || bLoadingGameplayMap)
    {
        return;
    }

    CancelInputRebind();
    bSettingsMenuOpen = false;
    bLoadingGameplayMap = true;

    if (UWorkOrderSubsystem* WorkOrderSubsystem = GetWorkOrderSubsystem())
    {
        WorkOrderSubsystem->PrepareSelectedOrderForLaunch();
    }

    ApplyMenuInputState();
    UpdateLobbyMusic();

    GetWorldTimerManager().ClearTimer(LoadingMapTimerHandle);
    if (LoadingScreenDelay <= 0.0f)
    {
        OpenGameplayMap();
        return;
    }

    GetWorldTimerManager().SetTimer(
        LoadingMapTimerHandle,
        this,
        &AThiefPlayerController::OpenGameplayMap,
        LoadingScreenDelay,
        false);
}
```

Рисунок 4.6 - Запуск перехода из lobby на игровую карту

Также в проекте есть обратный переход в стартовое *lobby*. Для этого используется функция *ReturnToLobby* в классе *AThiefPlayerController*. Она сбрасывает состояние загрузки, закрывает открытые меню, очищает ссылку на ближайшего торговца и открывает карту *NewMap*.

Функция *ReturnToLobby* полезна при тестировании. Разработчик может запустить игровую карту, проверить нужные механики, вернуться в *lobby* и снова начать проверку без полного перезапуска проекта.

Важным элементом системы является управление состояниями меню. Для этого используются переменные *bTradeMenuOpen*,

bProgressionMenuOpen, *bPauseMenuOpen*, *bSettingsMenuOpen* и *bLoadingGameplayMap*. Они определяют, какой экран должен отображаться и какие действия доступны пользователю.

Функция *ApplyMenuInputState* переключает режим ввода в зависимости от состояния проекта. В меню пользователь работает с кнопками и курсором, а на игровой карте управление передаётся персонажу.

Таким образом, стартовое *lobby* и переход на игровую карту реализованы через несколько связанных частей проекта. *DefaultEngine.ini* задаёт стартовую карту, *AThiefCatcerGameMode* подключает нужные классы и отключает создание *Pawn* в *lobby*, *AThiefPlayerController* управляет переходами и состояниями, а *APlayerGameHUD* отображает меню и *loading screen*.

4.3 Добавление ходьбы, управления камерой и анимаций

Одним из основных этапов разработки стало создание игрового персонажа. Через него пользователь взаимодействует с игровым миром: перемещается по карте, использует *sprint*, расходует и восстанавливает *stamina*, атакует руду, получает ресурсы, золото и опыт.

Логика персонажа разделена между двумя классами: *AGameplayCharacterBase* и *AThiefCatcher*. Класс *AGameplayCharacterBase* наследуется от *ACharacter* и хранит базовую игровую логику, а класс *AThiefCatcher* расширяет её управлением, *camera* и функциями движения.

На рисунке 4.7 показана структура классов игрового персонажа и их связь между собой.

Класс *AGameplayCharacterBase* содержит основные характеристики персонажа. В нём объявлены переменные *WalkSpeed*, *SprintSpeed*, *MaxStamina*, *Stamina*, *CurrentStamina*, *CollectedOreResources*, *CollectedGold*,

StaminaPotionCount, PlayerLevel, CurrentExperience, ExperienceToNextLevel и *AvailableUpgradePoints*.

```
#include "GameFramework/Character.h"
#include "GameplayCharacterBase.generated.h"
UCLASS()
class MYPROJECT_API AGameplayCharacterBase : public
ACharacter{GENERATED_BODY()
public:
    AGameplayCharacterBase();
    virtual void BeginPlay() override;
    virtual void EndPlay(const EEndPlayReason::Type EndPlayReason) override;
    virtual void SetupPlayerInputComponent(class UInputComponent*
PlayerInputComponent) override;
    virtual void Tick(float DeltaTime) override;
    UFUNCTION(BlueprintCallable, Category = "Attack")
    virtual void Attack();
    UFUNCTION(BlueprintCallable, Category = "Movement")
    void SetSprintActive(bool bShouldSprint);
    UFUNCTION(BlueprintCallable, Category = "Inventory")
    bool UseStaminaPotion();};
#include "Camera/CameraComponent.h"
#include "GameFramework/SpringArmComponent.h"
#include "GameplayCharacterBase.h"
#include "ThifCatcher.generated.h"
UCLASS()
class MYPROJECT_API AThifCatcher : public
AGameplayCharacterBase{GENERATED_BODY()
public:
    AThifCatcher();
    UPROPERTY(VisibleAnywhere)
    USpringArmComponent* SpringArm;
    UPROPERTY(VisibleAnywhere)
    UCameraComponent* Camera;
    virtual void SetupPlayerInputComponent(class UInputComponent*
PlayerInputComponent) override;
    void MoveForwardBackward(float Value);
    void MoveRightLeft(float Value);
    void Jump();
    void StopJump();
    void Sprint();
    void StopSprint();};
```

Рисунок 4.7 - Структура классов игрового персонажа

К основным параметрам персонажа относятся:

- скорость обычного передвижения через переменную *WalkSpeed*;
- скорость ускоренного движения через переменную *SprintSpeed*;
- текущее и максимальное значение *stamina*;
- количество добытой руды и золота;
- количество зелий восстановления *stamina*;
- уровень персонажа, текущий опыт и очки улучшений;
- урон по руде и параметры атаки.

В конструкторе *AGameplayCharacterBase* задаются начальные значения персонажа. Обычная скорость движения равна 450, скорость *sprint* равна 700, максимальная *stamina* равна 100, а начальный уровень персонажа равен 1. Также задаются параметры восстановления *stamina*, урона по руде, дальности атаки и времени атаки.

В этом же конструкторе подключаются ресурсы для атаки. Через *ConstructorHelpers* загружается *attack montage* и модель кирки. Кирка хранится в переменной *EquippedPickaxeMesh*, а для прикрепления используется *socket LeftHandSocket*.

На рисунке 4.8 показаны компоненты *camera* и параметры управляемого персонажа в классе *AThifCatcher*.

```
SpringArm =
CreateDefaultSubobject<USpringArmComponent>(TEXT("SpringArmComponent"));
SpringArm->SetupAttachment(RootComponent);
SpringArm->TargetArmLength = 420.0f;
SpringArm->SetRelativeLocation(FVector(0.0f, 0.0f, 70.0f));
SpringArm->bEnableCameraLag = true;
SpringArm->CameraLagSpeed = 12.0f;
SpringArm->bUsePawnControlRotation = true;
SpringArm->bDoCollisionTest = true;

Camera = CreateDefaultSubobject<UCameraComponent>(TEXT("MainCamera"));
Camera->SetupAttachment(SpringArm);
Camera->bUsePawnControlRotation = false;
Camera->SetRelativeRotation(FRotator(-10.0f, 0.0f, 0.0f));
|
```

Рисунок 4.8 - Компоненты камеры и *SpringArm* в классе *AThifCatcher*

Класс *AThifCatcher* отвечает за управляемого персонажа. В его конструкторе создаётся компонент *SpringArm*, который прикрепляется к корневому компоненту, получает длину 420 и смещение по высоте. Для него включены задержка *camera*, задержка поворота и использование поворота от *controller*.

После создания *SpringArm* создаётся компонент *Camera*. Он прикрепляется к *SpringArm* и получает небольшой наклон вниз. Благодаря этому *camera* располагается позади персонажа и показывает пространство перед ним.

В классе *AThifCatcher* также настраивается компонент движения. В нём задаются поворот в сторону движения, скорость поворота, сила прыжка, управление в воздухе, торможение и ускорение. Эти параметры делают управление более удобным на карте *Showcase*.

Функция *SetupPlayerInputComponent* отвечает за привязку действий пользователя к функциям персонажа. В *AGameplayCharacterBase* через неё подключаются действия *Attack* и *UsePotion*, а в *AThifCatcher* дополнительно подключаются движение, поворот *camera*, прыжок и *sprint*.

К основным функциям управления в классе *AThifCatcher* относятся:

- *MoveForwardBackward*, которая отвечает за движение вперёд и назад;
- *MoveRightLeft*, которая отвечает за движение вправо и влево;
- *Jump*, которая запускает прыжок персонажа;
- *StopJump*, которая завершает удержание прыжка;
- *Sprint*, которая включает ускоренное движение;
- *StopSprint*, которая отключает ускоренное движение.

Функции *MoveForwardBackward* и *MoveRightLeft* используют поворот *controller*, чтобы определить направление движения относительно *camera*. Поэтому персонаж движется не по глобальным осям карты, а относительно направления взгляда игрока.

Функция *Jump* вызывает стандартный прыжок персонажа через *ACharacter*. Если для прыжка назначен *animation montage*, он также может быть проигран. Функция *StopJump* вызывает *StopJumping* и завершает действие прыжка.

Функции *Sprint* и *StopSprint* не меняют скорость напрямую. Они вызывают *SetSprintActive* из *AGameplayCharacterBase*, потому что логика *sprint* связана с запасом *stamina* и должна храниться в базовом классе.

Работа *stamina* обновляется в функции *Tick*. Она вызывается каждый кадр и проверяет, движется ли персонаж с активным *sprint*. Если ускорение

используется, значение *stamina* уменьшается, а после остановки постепенно восстанавливается.

На рисунке 4.9 показаны основные параметры персонажа в классе *AGameplayCharacterBase*.

```
WalkSpeed = 450.0f;
SprintSpeed = 700.0f;
MaxStamina = 100.0f;
Stamina = MaxStamina;
MinusStamina = 32.0f;
PlusStamina = 26.0f;
MovingStaminaRegen = 2.5f;
StaminaRegenDelay = 0.85f;

OreDamage = 50.0f;
AttackRange = 350.0f;
AttackCooldown = 0.45f;

BaseExperienceToNextLevel = 100;
MaxStaminaUpgradeAmount = 20.0f;
OreDamageUpgradeAmount = 10.0f;
MoveSpeedUpqgradeAmount = 30.0f;
```

Рисунок 4.9 - Основные параметры персонажа в классе *AGameplayCharacterBase*

Функция *Attack* запускает удар киркой. Перед началом атаки проверяется, не выполняется ли атака уже сейчас, доступна ли она, не открыто ли меню и не находится ли персонаж в падении. Если условие не подходит, действие не запускается.

Когда атака начинается, персонаж выходит из *sprint*, останавливает прыжок и временно блокирует движение. Затем проигрывается *attack montage*, а момент нанесения урона вызывается через *timer*, чтобы удар совпадал с визуальным движением кирки.

Функция *UseStaminaPotion* отвечает за использование зелья восстановления *stamina*. Она проверяет наличие зелья и необходимость восстановления, затем уменьшает количество зелий, увеличивает *stamina*, обновляет *HUD* и сохраняет данные персонажа.

Для развития персонажа используются функции *AddExperience* и *SpendUpgradePoint*. Первая добавляет опыт и повышает уровень при

достижении нужного значения, а вторая тратит очко улучшения на выбранный параметр.

В классе также есть функции получения данных для *HUD*. Например, *GetCollectedGold*, *GetPlayerLevel*, *GetCurrentExperienceAmount*, *GetAvailableUpgradePoints* и *GetStaminaPotionCount* позволяют безопасно передавать значения в экранный *interface*.

При запуске персонажа вызывается *BeginPlay*. В ней загружаются данные из *JSON*, обновляются ресурсы и *stamina*, синхронизируется визуальная модель и прикрепляется кирка. При завершении работы вызывается *EndPlay*, где данные персонажа сохраняются.

Таким образом, игровой персонаж объединяет управление, *camera*, атаку, *stamina*, ресурсы, прогресс, зелья и сохранение данных. *AGameplayCharacterBase* отвечает за базовую игровую логику, а *AThifCatcher* добавляет управление и *camera*.

4.4 Реализация управления, камеры и системы *stamina*

После создания игрового персонажа была реализована система управления, *camera* от третьего лица и логика *stamina*. Эти элементы напрямую влияют на удобство игры, потому что пользователь должен свободно перемещаться по карте, видеть окружение, включать *sprint* и понимать, когда запас *stamina* заканчивается.

Основная часть управления находится в классе *AThifCatcher*. Он наследуется от *AGameplayCharacterBase* и добавляет компоненты, необходимые управляемому персонажу: *SpringArm*, *Camera* и функции движения. Базовая логика *stamina* находится в *AGameplayCharacterBase*, потому что она связана с состоянием персонажа, *HUD*, сохранениями и прокачкой.

Camera реализована через связку *SpringArm* и *Camera*. Компонент *SpringArm* создаётся в конструкторе *AThifCatcher*, прикрепляется к корневому компоненту персонажа и держит *camera* на расстоянии. В проекте длина рычага равна 420, а смещение по высоте равно 70.

Для *SpringArm* включены параметры плавности. Задержка движения и поворота делает *camera* менее резкой, а использование поворота от *controller* позволяет игроку свободно осматривать окружение.

Компонент *Camera* создаётся после *SpringArm* и прикрепляется к нему. Для него задан небольшой наклон вниз, чтобы игрок видел не только персонажа, но и пространство перед ним, включая руду, торговца и другие объекты на карте *Showcase*.

На рисунке 4.10 показана настройка камеры персонажа через *SpringArm* и *Camera*.

```
SpringArm =
CreateDefaultSubobject<USpringArmComponent>(TEXT("SpringArmComponent"));
SpringArm->SetupAttachment(RootComponent);
SpringArm->TargetArmLength = 420.0f;
SpringArm->SetRelativeLocation(FVector(0.0f, 0.0f, 70.0f));
SpringArm->bEnableCameraLag = true;
SpringArm->CameraLagSpeed = 12.0f;
SpringArm->bUsePawnControlRotation = true;
SpringArm->bDoCollisionTest = true;

Camera = CreateDefaultSubobject<UCameraComponent>(TEXT("MainCamera"));
Camera->SetupAttachment(SpringArm);
Camera->bUsePawnControlRotation = false;
Camera->SetRelativeRotation(FRotator(-10.0f, 0.0f, 0.0f));
```

Рисунок 4.10 - Настройка камеры персонажа в классе *AThifCatcher*

В конструкторе *AThifCatcher* также настраивается движение. У персонажа отключено прямое вращение от *controller* по осям *pitch*, *yaw* и *roll*. Вместо этого включён поворот в сторону движения, поэтому модель разворачивается по направлению фактического перемещения.

Компонент движения получает параметры прыжка и перемещения. *JumpZVelocity* отвечает за силу прыжка, *AirControl* позволяет управлять персонажем в воздухе, *BrakingDecelerationWalking* задаёт торможение, а *MaxAcceleration* отвечает за ускорение.

Основные параметры движения персонажа:

- *bOrientRotationToMovement* разворачивает персонажа в сторону движения;
- *RotationRate* задаёт скорость поворота персонажа;
- *JumpZVelocity* определяет силу прыжка;
- *AirControl* позволяет управлять персонажем в воздухе;
- *BrakingDecelerationWalking* отвечает за торможение после остановки;
- *MaxWalkSpeed* задаёт текущую скорость передвижения.

Управление связано с настройками *input* из файла *DefaultInput.ini*. Для движения используются клавиши *W*, *A*, *S*, *D*, для поворота *camera* используются *MouseX* и *MouseY*, прыжок выполняется через *Space Bar*, а *sprint* включается через *Left Shift*.

Функция *SetupPlayerInputComponent* связывает настройки *input* с функциями персонажа. Ось *MoveForwardBackward* вызывает *MoveForwardBackward*, ось *MoveRightLeft* вызывает *MoveRightLeft*, ось *Turn* отвечает за горизонтальный поворот, а *lookUpDown* отвечает за вертикальный обзор.

Клавиши и оси управления используются следующим образом:

- *W* и *S* вызывают движение вперёд и назад через *MoveForwardBackward*;
- *A* и *D* вызывают движение влево и вправо через *MoveRightLeft*;
- *MouseX* отвечает за поворот *camera* по горизонтали;
- *MouseY* отвечает за поворот *camera* по вертикали;
- *Space Bar* вызывает *Jump* и *StopJump*;
- *Left Shift* вызывает *Sprint* и *StopSprint*.

На рисунке 4.11 показаны функции привязки управления персонажа.

Функция *MoveForwardBackward* отвечает за движение вперёд и назад. Сначала она проверяет, не находится ли персонаж в состоянии атаки, существует ли *controller* и не равно ли значение оси нулю. Если атака активна, движение временно блокируется.


```

void AThifCatcher::SetupPlayerInputComponent(UInputComponent*
PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

    PlayerInputComponent->BindAxis(TEXT("MoveForwardBackward"), this,
&AThifCatcher::MoveForwardBackward);
    PlayerInputComponent->BindAxis(TEXT("MoveRightLeft"), this,
&AThifCatcher::MoveRightLeft);

    PlayerInputComponent->BindAxis(TEXT("Turn"), this,
&AThifCatcher::AddControllerYawInput);
    PlayerInputComponent->BindAxis(TEXT("lookUpDown"), this,
&AThifCatcher::AddControllerPitchInput);

    PlayerInputComponent->BindAction(TEXT("Jump"), IE_Pressed, this,
&AThifCatcher::Jump);
    PlayerInputComponent->BindAction(TEXT("Jump"), IE_Released, this,
&AThifCatcher::StopJump);

    PlayerInputComponent->BindAction(TEXT("Sprint"), IE_Pressed, this,
&AThifCatcher::Sprint);
    PlayerInputComponent->BindAction(TEXT("Sprint"), IE_Released, this,
&AThifCatcher::StopSprint);
}

```

Рисунок 4.11 - Привязка input к функциям движения, прыжка и sprint

После проверки функция получает текущий поворот *controller*. Для движения используется только горизонтальное направление, чтобы перемещение по земле не зависело от наклона *camera*. Затем направление передаётся в *AddMovementInput*.

Функция *MoveRightLeft* работает похожим образом, но отвечает за боковое движение. Она получает направление вправо или влево относительно поворота *controller*, поэтому управление остаётся привычным для игры от третьего лица.

Прыжок реализован через функции *Jump* и *StopJump*. Функция *Jump* проверяет состояние атаки и при свободном персонаже вызывает стандартный прыжок из *ACharacter*. Функция *StopJump* завершает удержание прыжка после отпускания клавиши.

Функции *Sprint* и *StopSprint* отвечают за включение и отключение ускоренного движения. Они не меняют скорость напрямую, а вызывают *SetSprintActive* из *AGameplayCharacterBase*, потому что *sprint* должен учитывать запас *stamina*.

Функция *SetSprintActive* принимает значение, которое включает или выключает ускорение. Если игрок пытается включить *sprint*, но *stamina* закончилась, ускорение не запускается. Если запас есть, скорость переключается между *WalkSpeed* и *SprintSpeed*.

Система *stamina* обновляется в функции *Tick* класса *AGameplayCharacterBase*. Она вызывается каждый кадр и проверяет, есть ли у игрока намерение двигаться. Для этого используется функция *HasMovementInputIntent*.

Для расхода *stamina* недостаточно удерживать *Left Shift*. Персонаж должен одновременно находиться в состоянии *sprint* и реально двигаться. Поэтому *stamina* не тратится, если игрок стоит на месте.

На рисунке 4.12 показана логика расхода и восстановления *stamina*. Если персонаж движется с включённым *sprint*, вызывается функция *DecreaseStamina*.

```
void AGameplayCharacterBase::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

    const bool bHasMovementIntent = HasMovementInputIntent();
    const bool bIsActivelySprinting = bIsSprint && bHasMovementIntent;

    if (bIsActivelySprinting && Stamina > MinimumSprintStamina)
    {
        DecreaseStamina();
    }
    else
    {
        TimeSinceLastStaminaUse += DeltaTime;
        if (!bIsSprint && !FMath::IsNearlyEqual(Stamina, MaxStamina) &&
            TimeSinceLastStaminaUse >= StaminaRegenDelay)
        {
            IncreaseStamina();
        }
    }
}
```

Рисунок 4.12 - Обновление *stamina* в функции *Tick*

Она уменьшает значение с учётом времени между кадрами и переменной *MinusStamina*, ограничивает значение допустимыми границами и

обновляет *HUD*. Если *sprint* не активен, функция *Tick* увеличивает время после последнего расхода. После задержки *StaminaRegenDelay* начинается восстановление через *IncreaseStamina*. В покое восстановление идёт быстрее, а при обычном движении медленнее.

Основные правила работы *stamina*:

- *stamina* расходуется только при активном *sprint* и наличии движения;
- при нулевом значении *stamina* ускорение автоматически отключается;
- восстановление начинается после задержки *StaminaRegenDelay*;
- при обычном движении восстановление медленнее, чем в состоянии покоя;
- после каждого изменения значения обновляется панель *HUD*.

Связь с *HUD* является важной частью системы. Пользователь должен видеть, сколько *stamina* осталось, поэтому после расхода или восстановления вызывается *UpdateStaminaBar*, которая передаёт актуальные значения в *interface*.

В результате управление, *camera* и *stamina* работают как единая система. *AThifCatcher* отвечает за *input*, движение, *camera*, прыжок и *sprint*, а *AGameplayCharacterBase* отвечает за состояние персонажа, расход и восстановление *stamina*, изменение скорости и обновление *HUD*.

4.5 Создание рудных объектов

После настройки управления и системы *stamina* была реализована механика атаки. В проекте она используется не для боя с противниками, а для добычи руды. Игрок подходит к рудному объекту, нажимает кнопку атаки, персонаж выполняет удар киркой, а затем код проверяет цель перед персонажем. Основная логика атаки находится в классе *AGameplayCharacterBase*. За запуск отвечает функция *Attack*, которая

вызывается при нажатии *LeftMouseButton*, связанной с действием *Attack* в настройках *input*.

На рисунке 4.13 показана функция запуска атаки персонажа.

```
void AGameplayCharacterBase::Attack()
{
    if (bIsAttacking || !bCanAttack)
    {
        return;
    }

    bCanAttack = false;
    bIsAttacking = true;
    SetSprintActive(false);
    StopJumping();

    if (AController* CurrentController = GetController())
    {
        CurrentController->SetIgnoreMoveInput(true);
    }

    if (UCharacterMovementComponent* MovementComponent =
    GetCharacterMovement())
    {
        MovementComponent->StopMovementImmediately();
        MovementComponent->DisableMovement();
    }

    GetWorldTimerManager().SetTimer(
        AttackHitTimerHandle,
        this,
        &AGameplayCharacterBase::TriggerAttackHit,
        FMath::Min(EffectiveHitDelay, EffectiveAttackDuration),
        false);

    GetWorldTimerManager().SetTimer(
        AttackStateTimerHandle,
        this,
        &AGameplayCharacterBase::FinishAttackState,
        EffectiveAttackDuration,
        false);
}
```

Рисунок 4.13 - Запуск атаки персонажа в классе *AGameplayCharacterBase*

Важной частью атаки является задержка между началом *animation* и моментом нанесения урона. Урон не наносится сразу после нажатия кнопки. Для этого используется параметр *AttackHitNormalizedTime*, который определяет момент попадания внутри анимации.

В функции *Attack* запускаются два *timers*. Первый вызывает *TriggerAttackHit* и отвечает за момент удара. Второй вызывает *FinishAttackState* и завершает состояние атаки, чтобы вернуть персонажу управление. Функция *TriggerAttackHit* не выполняет всю механику сама, а вызывает *TryDamageOre*. Благодаря этому момент срабатывания удара отделён от поиска рудного объекта, поэтому код проще читать и дорабатывать.

На рисунке 4.14 показана связь между моментом удара и проверкой рудного объекта.

```
void AGameplayCharacterBase::TryDamageOre(){
    UWorld* World = GetWorld();
    if (!World){return;}
    auto ResolveOreFromHit = [](const FHitResult& HitResult) -> AOreStoneBase*{
        if (AOreStoneBase* OreStone = Cast<AOreStoneBase>(HitResult.GetActor())){return OreStone;}
        if (UPrimitiveComponent* HitComponent = HitResult.GetComponent()){
            if (AOreStoneBase* OreStoneOwner = Cast<AOreStoneBase>(HitComponent->GetOwner())){return OreStoneOwner;}}
        return nullptr;};
    auto ApplyOreDamage = [this](AOreStoneBase* OreStone){
        if (!OreStone || !OreStone->IsOreAvailable()){return false;}
        const bool bWillBreak = OreStone->GetCurrentHealth() <= OreDamage;
        OreStone->ApplyDamageToOre(OreDamage);
        if (bWillBreak && !OreStone->IsOreAvailable()){
            TotalOreNodesBroken += 1;
            AddOreResources(OreStone->GetOreResourceReward());
            AddExperience(OreStone->GetOreExperienceReward());return true;};
    if (APlayerController* PlayerController = Cast<APlayerController>(GetController()))
    {FVector TraceStart;
    FRotator TraceRotation;
    PlayerController->GetPlayerViewPoint(TraceStart, TraceRotation);
    const FVector TraceEnd = TraceStart + (TraceRotation.Vector() * AttackRange);
    FCollisionQueryParams QueryParams(SCENE_QUERY_STAT(OreAttackTrace), false, this);
    QueryParams.AddIgnoredActor(this);
    const FCollisionShape AttackShape = FCollisionShape::MakeSphere(65.0f);
    TArray<FHitResult> HitResults;
    if (World->SweepMultiByChannel(HitResults, TraceStart, TraceEnd, FQuat::Identity, ECC_Visibility, AttackShape, QueryParams))
    {for (const FHitResult& HitResult : HitResults)
        {if (ApplyOreDamage(ResolveOreFromHit(HitResult)))return;}}
    const FVector CharacterLocation = GetActorLocation();
    FVector AttackDirection = GetActorForwardVector();
    if (AController* CurrentController = GetController()){
        FRotator ControlRotation = CurrentController->GetControlRotation();
        ControlRotation.Pitch = 0.0f;
        ControlRotation.Roll = 0.0f;
        AttackDirection = ControlRotation.Vector().GetSafeNormal();}
    TArray<AActor*> OreActors;
    UGameplayStatics::GetAllActorsOfClass(World, AOreStoneBase::StaticClass(), OreActors);
    AOreStoneBase* BestOreTarget = nullptr;
    float BestScore = -FLT_MAX;
    const float CloseRangeOverride = FMath::Min(AttackRange, 220.0f);
    for (AActor* OreActor : OreActors){
        AOreStoneBase* OreStone = Cast<AOreStoneBase>(OreActor);
        if (!OreStone || !OreStone->IsOreAvailable()){continue;}
        const FVector ToOre = OreStone->GetActorLocation() - CharacterLocation;
        const float DistanceToOre = ToOre.Size();
        if (DistanceToOre > AttackRange || DistanceToOre <= KINDA_SMALL_NUMBER){continue;}
        const FVector ToOreDirection = ToOre.GetSafeNormal();
        const float FacingDot = FVector::DotProduct(AttackDirection, ToOreDirection);
        const bool bIsVeryCloseOre = DistanceToOre <= CloseRangeOverride;
        if (!bIsVeryCloseOre && FacingDot < 0.15f){continue;}
        const float Score = bIsVeryCloseOre
            ? 10.0f - (DistanceToOre / CloseRangeOverride)
            : (FacingDot * 2.0f) - (DistanceToOre / AttackRange);
        if (Score > BestScore){
            BestScore = Score;
            BestOreTarget = OreStone;}}
    ApplyOreDamage(BestOreTarget);}
```

Рисунок 4.14 - Проверка рудного объекта во время атаки

Функция *TryDamageOre* ищет руду, по которой может попасть персонаж. Она получает положение персонажа, направление взгляда и

проверяет объекты в пределах дистанции *AttackRange*. В проекте эта дистанция равна 350 единицам.

При проверке цели учитывается не только расстояние, но и направление. Руда должна находиться перед персонажем, а не сбоку или позади него. Если подходящая цель найдена, ей передаётся урон через функцию *ApplyDamageToOre*.

После нанесения урона проверяется состояние руды. Если объект был разрушен, персонаж получает ресурс через *AddOreResources* и опыт через *AddExperience*. Количество ресурса и опыта берётся из самого объекта руды.

Основной порядок добычи руды выглядит так:

- игрок нажимает кнопку атаки;
- персонаж запускает *attack montage*;
- во время *animation* срабатывает момент удара;
- функция *TryDamageOre* ищет доступную руду перед персонажем;
- найденной руде наносится урон;
- после разрушения руды персонаж получает ресурс и опыт.

Функция *FinishAttackState* завершает атаку. Она снимает флаг активной атаки, возвращает движение и снова устанавливает обычную скорость или скорость *sprint* в зависимости от текущего состояния.

Также отдельно работает задержка между атаками. Она не позволяет игроку слишком часто нажимать кнопку удара и делает добычу более контролируемой.

На рисунке 4.15 показано завершение атаки и возвращение управления персонажу.

Таким образом, атака состоит из нескольких связанных частей. Класс *AGameplayCharacterBase* запускает удар, проигрывает *attack montage*, блокирует движение, выбирает момент попадания, ищет руду и начисляет награду. Класс *AOreStoneBase* принимает урон, хранит здоровье объекта и определяет, разрушена ли руда.

```

void AGameplayCharacterBase::FinishAttackState()
{
    bIsAttacking = false;

    if (UCharacterMovementComponent* MovementComponent =
        GetCharacterMovement())
    {
        MovementComponent->SetMovementMode(MOVE_Walking);
        MovementComponent->MaxWalkSpeed = bIsSprint ? SprintSpeed :
WalkSpeed;
    }

    if (AController* CurrentController = GetController())
    {
        CurrentController->SetIgnoreMoveInput(false);
    }

    SyncAttachedVisualInputState();
}

```

Рисунок 4.15 - Завершение атаки и возврат управления персонажу

4.6 Добыча ресурсов и интерфейс

Для добычи ресурсов в проекте создан отдельный класс рудного объекта *AOreStoneBase*. Он наследуется от *AActor*, поэтому руду можно размещать на карте *Showcase* как самостоятельный игровой объект. Руда имеет модель, здоровье, панель здоровья, область показа панели, награду за разрушение и систему *respawn*.

В конструкторе *AOreStoneBase* создаются основные компоненты объекта. Корневым компонентом является *SceneRoot*. К нему прикрепляется *OreMesh*, который отвечает за визуальное отображение руды. Также создаются *HealthBarWidgetComponent* и *HealthBarTriggerSphere*.

Основные компоненты рудного объекта:

- *SceneRoot* используется как корневой компонент руды;
- *OreMesh* отвечает за визуальное отображение рудного объекта;
- *HealthBarWidgetComponent* отображает здоровье руды на экране;
- *HealthBarTriggerSphere* определяет, находится ли персонаж рядом;

– *RespawnTimerHandle* используется для восстановления руды после разрушения.

На рисунке 4.16 показана структура класса *AOreStoneBase* и его основные компоненты.

```
SceneRoot = CreateDefaultSubobject<USceneComponent>(TEXT("SceneRoot"));
SetRootComponent(SceneRoot);

OreMesh = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("OreMesh"));
OreMesh->SetupAttachment(SceneRoot);

HealthBarWidgetComponent =
CreateDefaultSubobject<UWidgetComponent>(TEXT("HealthBarWidget"));
HealthBarWidgetComponent->SetupAttachment(SceneRoot);
HealthBarWidgetComponent->SetWidgetSpace(EWidgetSpace::Screen);
HealthBarWidgetComponent->SetWidgetClass(UOreHealthBarWidget::StaticClass());
HealthBarWidgetComponent->SetVisibility(false);

HealthBarTriggerSphere =
CreateDefaultSubobject<USphereComponent>(TEXT("HealthBarTriggerSphere"));
HealthBarTriggerSphere->SetupAttachment(SceneRoot);
```

Рисунок 4.16 - Компоненты рудного объекта в классе *AOreStoneBase*

У руды есть параметры здоровья. Переменная *MaxHealth* хранит максимальное значение, а *CurrentHealth* хранит текущее значение. При создании объекта текущее здоровье устанавливается равным максимальному. Функция *GetHealthPercent* возвращает процент здоровья для отображения в *widget*.

Функция *ApplyDamageToOre* вызывается, когда персонаж попадает по руде атакой. Она проверяет, доступна ли руда и больше ли значение урона нуля. Если объект уже разрушен, урон не применяется. Если руда доступна, текущее здоровье уменьшается.

Функция *BreakOre* переводит руду в разрушенное состояние. Она отключает доступность объекта, устанавливает здоровье в ноль, скрывает модель, отключает *collision* и прячет панель здоровья. После этого запускается *timer*, который вызывает *RespawnOre*.

На рисунке 4.17 показана логика получения урона и разрушения руды.


```

void AOreStoneBase::ApplyDamageToOre(float DamageAmount)
{
    if (!bOreAvailable || DamageAmount <= 0.0f){return;}
    CurrentHealth = FMath::Clamp(CurrentHealth - DamageAmount, 0.0f,
MaxHealth);
    if (CurrentHealth <= 0.0f){
        BreakOre();
        return;}
    RefreshHealthBar();
    UpdateHealthBarVisibility();}
void AOreStoneBase::BreakOre(){
    bOreAvailable = false;
    CurrentHealth = 0.0f;
    if (OreMesh){
        OreMesh->SetVisibility(false, true);
        OreMesh->SetCollisionEnabled(ECollisionEnabled::NoCollision);}
    GetWorldTimerManager().SetTimer(
        RespawnTimerHandle,
        this,
        &AOreStoneBase::RespawnOre,
        RespawnDelayMinutes * 60.0f,
        false);}

```

Рисунок 4.17 - Получение урона и разрушение рудного объекта

Система *respawn* нужна для повторного появления руды после разрушения. Задержка восстановления задаётся переменной *RespawnDelayMinutes*. Значение хранится в минутах, а при запуске *timer* переводится в секунды.

Функция *RespawnOre* возвращает руду в рабочее состояние. Она снова делает объект доступным, восстанавливает здоровье до *MaxHealth*, включает видимость модели и возвращает *collision*. После этого обновляется панель здоровья и проверяется, находится ли игрок рядом.

Отображение здоровья связано с *HealthBarWidgetComponent*. Этот компонент использует класс *UOreHealthBarWidget*. Функция *RefreshHealthBar* передаёт в *widget* текущее и максимальное здоровье, чтобы игрок видел состояние руды.

Панель здоровья не отображается постоянно. За это отвечает *HealthBarTriggerSphere*. Когда персонаж входит в радиус руды, вызывается обработчик начала пересечения, а когда выходит из радиуса, вызывается

обработчик окончания пересечения. После этого обновляется видимость панели.

Условия отображения панели здоровья руды:

- руда должна быть доступна;
- персонаж должен находиться в радиусе *HealthBarTriggerSphere*;
- не должно быть открыто модальное меню;
- игра не должна находиться в состоянии загрузки;
- объект должен иметь корректный *HealthBarWidgetComponent*.

На рисунке 4.18 показана настройка панели здоровья и зоны её отображения.

```
void AOreStoneBase::RefreshHealthBar()
{if (!HealthBarWidgetComponent){return;}
  HealthBarWidgetComponent->InitWidget();
  if (UOreHealthBarWidget* HealthBarWidget =
    Cast<UOreHealthBarWidget>(HealthBarWidgetComponent-
  >GetUserWidgetObject())){
    HealthBarWidget->SetHealthValues(CurrentHealth, MaxHealth);}}
void AOreStoneBase::UpdateHealthBarVisibility(){
  if (!HealthBarWidgetComponent){return;}
  if (!bOreAvailable){
    HealthBarWidgetComponent->SetVisibility(false);
    return;}
  HealthBarWidgetComponent->SetVisibility(bPlayerInHealthBarRange);}
void AOreStoneBase::HandleHealthBarTriggerBeginOverlap(
  UPrimitiveComponent* OverlappedComponent,
  AActor* OtherActor,
  UPrimitiveComponent* OtherComp,
  int32 OtherBodyIndex,
  bool bFromSweep,
  const FHitResult& SweepResult){
  if (!bOreAvailable){return;}
  if (APawn* PlayerPawn = UGameplayStatics::GetPlayerPawn(this, 0)){
    if (OtherActor == PlayerPawn){
      bPlayerInHealthBarRange = true;
      RefreshHealthBar();
      UpdateHealthBarVisibility();}}}
```

Рисунок 4.18 - Отображение *HealthBarWidget* рядом с рудой

4.7 Добавление возможности прокачки персонажа

После реализации добычи руды была создана система торговца. Она делает добытый ресурс полезным: игрок может продавать руду, получать

золото и покупать зелье восстановления *stamina*. Таким образом добыча, экономика и развитие персонажа объединяются в один игровой цикл.

Объект торговца реализован в классе *ATraderNPC*. Он наследуется от *AActor* и размещается на карте. Внутри класса создаются компоненты *SceneRoot*, *TraderMesh*, *TraderCharacterMesh* и *InteractionSphere*. Компонент *InteractionSphere* определяет, находится ли персонаж рядом и может ли он открыть торговое меню.

В конструкторе *ATraderNPC* задаются начальные параметры торговца. Создаётся корневой компонент, подключается визуальная модель, задаётся масштаб и настраивается зона взаимодействия. Радиус *InteractionSphere* составляет 220 единиц, поэтому игрок должен подойти достаточно близко.

Структура торговца в классе *ATraderNPC* приеден на рисунке 4.19.

```
UCLASS()
class MYPROJECT_API ATraderNPC : public AAActor{
    GENERATED_BODY()
public:
    ATraderNPC();
    FText GetTraderName() const;
protected:
    virtual void BeginPlay() override;
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Components")
    TObjectPtr<USceneComponent> SceneRoot;
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Components")
    TObjectPtr<UStaticMeshComponent> TraderMesh;
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Components")
    TObjectPtr<USkeletalMeshComponent> TraderCharacterMesh;
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Interaction")
    TObjectPtr<USphereComponent> InteractionSphere;
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Interaction")
    FText TraderName;};
```

Рисунок 4.19 - Структура торговца в классе *ATraderNPC*

В функции *BeginPlay* торговец подключает обработчики пересечения зоны взаимодействия. При входе персонажа в *InteractionSphere* вызывается обработчик начала пересечения, при выходе — обработчик окончания пересечения.

Если в зону входит персонаж класса *AGameplayCharacterBase*, торговец получает его *controller*. Затем вызывается функция *SetNearbyTrader* в классе

AThiefPlayerController. Она сохраняет ссылку на ближайшего торговца, чтобы *HUD* мог показать подсказку, а *controller* разрешил открыть меню торговли.

При выходе персонажа из зоны взаимодействия вызывается функция *ClearNearbyTrader*. Она очищает ссылку на торговца и закрывает торговое меню, если оно было открыто. Это ограничивает торговлю расстоянием и связывает доступность действия с положением персонажа.

Основные действия торговца:

- определение приближения персонажа через *InteractionSphere*;
- передача ссылки на торговца в *controller*;
- отображение подсказки для игрока;
- открытие торгового меню рядом с торговцем;
- закрытие торгового меню при выходе из зоны взаимодействия.

На рисунке 4.24 показана обработка входа и выхода персонажа из зоны торговца.

```
void ATraderNPC::HandleInteractionSphereBeginOverlap(
    UPrimitiveComponent* OverlappedComponent,
    AActor* OtherActor,
    UPrimitiveComponent* OtherComp,
    int32 OtherBodyIndex,
    bool bFromSweep,
    const FHitResult& SweepResult){
    if (AGameplayCharacterBase* PlayerCharacter =
        Cast<AGameplayCharacterBase>(OtherActor)){
        if (AThiefPlayerController* ThiefController =
            Cast<AThiefPlayerController>(PlayerCharacter-
                >GetController())){
            ThiefController->SetNearbyTrader(this);}}
void ATraderNPC::HandleInteractionSphereEndOverlap(
    UPrimitiveComponent* OverlappedComponent,
    AActor* OtherActor,
    UPrimitiveComponent* OtherComp,
    int32 OtherBodyIndex){
    if (AGameplayCharacterBase* PlayerCharacter =
        Cast<AGameplayCharacterBase>(OtherActor)){
        if (AThiefPlayerController* ThiefController =
            Cast<AThiefPlayerController>(PlayerCharacter->GetController()))
            {ThiefController->ClearNearbyTrader(this);}}
```

Рисунок 4.20 - Проверка взаимодействия персонажа с торговцем

Торговое меню открывается через *AThiefPlayerController*. Для этого используется функция *ToggleTradeMenu*. Она проверяет, находится ли рядом торговец, не открыты ли другие меню и можно ли сейчас перейти в состояние

торговли. Сама торговля выполняется через функции персонажа. Продажа руды реализована функцией *SellOre*. Она принимает количество руды и цену за одну единицу. Если ресурсов недостаточно, операция не выполняется. Если руды хватает, её количество уменьшается, а золото увеличивается. Покупка зелья восстановления *stamina* реализована функцией *BuyStaminaPotion*. Она проверяет, достаточно ли у персонажа золота. Если золота хватает, оно уменьшается, количество зелий увеличивается, а данные персонажа сохраняются.

К торговым действиям проекта относятся:

- продажа одной единицы руды;
- продажа пяти единиц руды;
- покупка одного зелья восстановления *stamina*;
- обновление количества руды, золота и зелий;
- сохранение данных персонажа после успешной операции.

На рисунке 4.25 показана логика продажи руды и покупки зелья.

```
bool AGameplayCharacterBase::SellOre(const int32 OreAmount, const int32
GoldPerOre){
    if (OreAmount <= 0 || GoldPerOre <= 0 || CollectedOreResources <
OreAmount){return false;}
    CollectedOreResources -= OreAmount;
    const int32 GoldEarned = OreAmount * GoldPerOre;
    CollectedGold += GoldEarned;
    TotalGoldEarned += GoldEarned;
    TotalOreSold += OreAmount;
    UpdateResourceCounter();
    SaveCharacterDataToJson();
    return true;}
bool AGameplayCharacterBase::BuyStaminaPotion(const int32 GoldCost, const
float RestoreAmount){
    if (GoldCost <= 0 || RestoreAmount <= 0.0f || CollectedGold < GoldCost){
        return false;}
    CollectedGold -= GoldCost;
    StaminaPotionCount += 1;
    StaminaPotionRestoreAmount = RestoreAmount;
    TotalPotionsBought += 1;
    SaveCharacterDataToJson();
    return true;}
```

Рисунок 4.21 - Продажа руды и покупка зелья *stamina*

В классе *APlayerGameHUD* для торговли используется функция *DrawTradePanel*. Она отображает количество руды, золота и зелий, а также

кнопки торговых действий. Пользователь может продавать одну руду, продавать пять единиц руды или покупать зелье восстановления *stamina*.

Торговец связывает несколько систем проекта. Руда появляется после добычи, золото появляется после продажи, зелье восстанавливает *stamina*, а *stamina* позволяет дольше использовать *sprint*. Поэтому торговля не является отдельным декоративным окном, а поддерживает основной игровой цикл.

4.8 Инвентарь, зелья и меню персонажа

Для отображения информации на экране используется собственный *HUD*. Он реализован в классе *APlayerGameHUD* и отвечает за стартовое меню, экран загрузки, меню паузы, настройки, ресурсную панель, панель *stamina*, торговое меню и меню прокачки. В проекте *HUD* рисуется через *Canvas*. Такой подход позволяет вручную выводить панели, текст, кнопки, фоновые элементы и подсказки. Для учебного проекта это удобно, потому что основную логику отображения можно проследить в одном классе.

На рисунке 4.22 показана общая структура класса *APlayerGameHUD*.

```
UCLASS()
class MYPROJECT_API APlayerGameHUD : public AHUD{
    GENERATED_BODY()
public:
    virtual void DrawHUD() override;
    void HandleMenuClick(const FVector2D& MousePosition);
private:
    void DrawMainMenu(float ViewportWidth, float ViewportHeight);
    void DrawLoadingScreen(float ViewportWidth, float ViewportHeight);
    void DrawSettingsPanel(float ViewportWidth, float ViewportHeight, bool
bFromPauseMenu);
    void DrawPauseMenu(float ViewportWidth, float ViewportHeight);
    void DrawTradePanel(float ViewportWidth, float ViewportHeight);
    void DrawProgressionPanel(float ViewportWidth, float ViewportHeight);
    void DrawShiftResultScreen(float ViewportWidth, float ViewportHeight);
    void DrawResourcePanel(float ViewportWidth);
    void DrawLevelPanel();
    void DrawWorkOrderPanel(float ViewportWidth, float ViewportHeight);
    void DrawStaminaPanel(float ViewportHeight);
    void DrawInteractionPrompt(float ViewportWidth, float ViewportHeight);
    void DrawLevelUpPopup(float ViewportWidth, float ViewportHeight);};
```

Рисунок 4.22 - Структура HUD в классе *APlayerGameHUD*

Главная функция отрисовки проверяет текущее состояние игры и выбирает, какие элементы нужно показать. На карте *NewMap* отображается стартовое меню, при загрузке показывается *loading screen*, а на карте *Showcase* отображаются игровые панели, подсказки и активные меню.

Стартовое меню рисуется функцией *DrawMainMenu*. В нём отображается название проекта, краткое описание игрового цикла и кнопки запуска. Пользователь может начать игру, открыть настройки или выйти из проекта.

Экран загрузки рисуется функцией *DrawLoadingScreen*. Он появляется между стартовым *lobby* и игровой картой. На экране отображаются затемнённый фон, центральная панель, текст загрузки и полоса прогресса.

Меню паузы рисуется функцией *DrawPauseMenu*. Оно появляется во время игровой сцены и позволяет продолжить игру, открыть настройки, вернуться в *lobby* или выйти из проекта.

К основным экранам *HUD* относятся:

- стартовое меню на карте *NewMap*;
- *loading screen* перед переходом на *Showcase*;
- меню паузы на игровой карте;
- панель настроек звука, управления и *UI*;
- торговое меню рядом с торговцем;
- меню прокачки персонажа;
- игровые панели ресурсов, уровня и *stamina*.

На рисунке 4.23 показано стартовое меню или экран загрузки проекта.

Во время игры *HUD* отображает ресурсную панель. Она рисуется функцией *DrawResourcePanel*. На ней выводится количество руды, золота и зелий восстановления *stamina*. Эти данные берутся из персонажа через функции получения значений.

Панель *stamina* рисуется функцией *DrawStaminaPanel*. Она показывает текущее и максимальное значение *stamina*. При использовании *sprint* эта панель помогает игроку понимать, сколько ускорения ещё доступно.

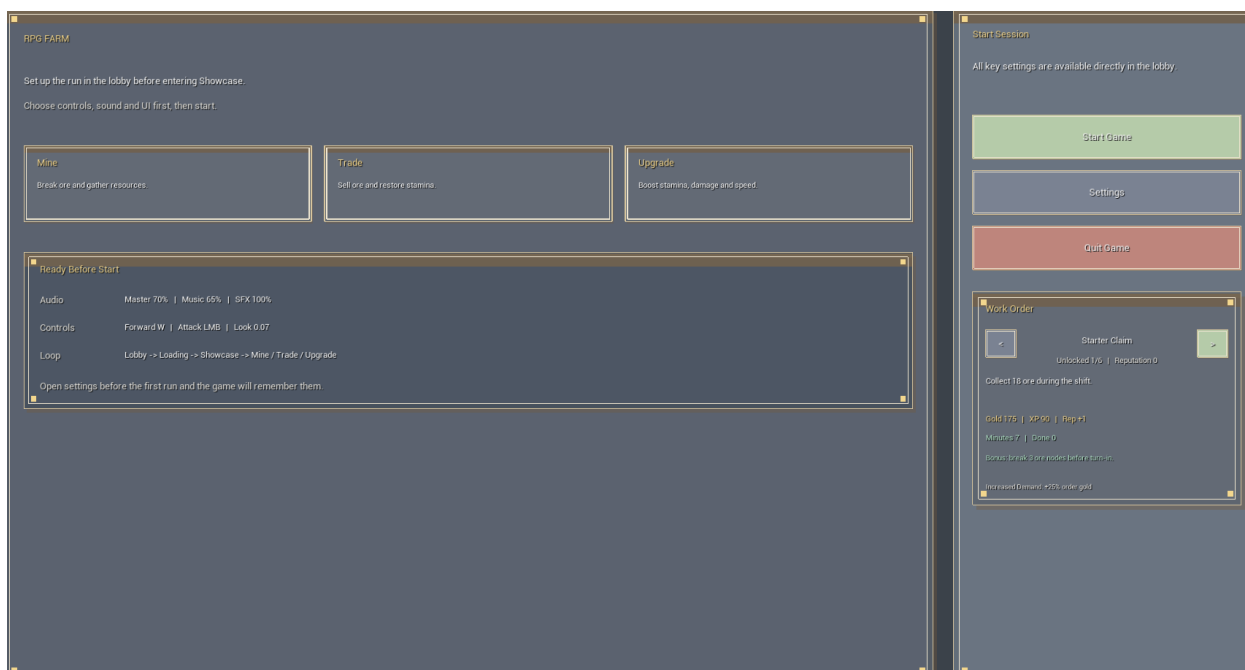


Рисунок 4.23 - Стартовое меню и экран загрузки проекта

Торговое меню рисуется функцией *DrawTradePanel*. Оно открывается рядом с торговцем и показывает доступные торговые действия. В нём отображаются руды, золото, зелья и кнопки продажи или покупки.

Меню прокачки рисуется функцией *DrawProgressionPanel*. Оно показывает уровень персонажа, текущий опыт, очки улучшений и основные характеристики. Через это меню игрок может усилить максимальную *stamina*, урон по руде или скорость движения.

На рисунке 4.24 показаны игровые панели *HUD* на карте *Showcase*.

Важной частью *HUD* является обработка кнопок. Для этого Важной частью *HUD* является обработка кнопок. Для этого используется перечисление *EHUDMenuAction*. Каждая кнопка получает своё действие, а

после нажатия *HUD* передаёт команду в *controller* или вызывает нужную игровую функцию.



Рисунок 4.24 - Игровые панели HUD на карте Showcase

Таким образом, *HUD* выполняет не только декоративную роль. Он связывает пользователя с запуском игры, настройками, торговлей, прокачкой, ресурсами и состоянием *stamina*. Класс *APlayerGameHUD* делает проект понятнее, потому что игрок видит нужные данные и может управлять системами через экранные элементы.

4.9 Торговец и торговля ресурсами

После реализации добычи и торговли была добавлена система прокачки персонажа. Она нужна для того, чтобы игрок видел развитие после действий на карте. Прокачка добавляет долгосрочную цель: получать опыт, повышать уровень и улучшать характеристики.

Основная логика прокачки находится в классе *AGameplayCharacterBase*. В нём хранятся переменные *PlayerLevel*, *CurrentExperience*, *ExperienceToNextLevel* и *AvailableUpgradePoints*. Они отвечают за уровень, текущий опыт, опыт до следующего уровня и доступные очки улучшений.

Начальные значения задаются в конструкторе. Персонаж начинает с первого уровня, нулевого опыта и без очков улучшений. Базовое значение опыта для следующего уровня задаётся переменной *BaseExperienceToNextLevel* и равно 100.

На рисунке 4.25 показаны переменные прогресса персонажа в классе *AGameplayCharacterBase*.

```
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 PlayerLevel;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 CurrentExperience;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 ExperienceToNextLevel;|
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 AvailableUpgradePoints;
```

Рисунок 4.25 - Параметры прогресса персонажа

Опыт добавляется через функцию *AddExperience*. Она принимает количество опыта, проверяет его значение и прибавляет к текущему прогрессу. После этого функция проверяет, достиг ли персонаж порога следующего уровня.

Если текущий опыт становится больше или равен требуемому значению, уровень повышается. Из текущего опыта вычитается нужное количество, переменная *PlayerLevel* увеличивается, а персонаж получает одно очко улучшения. После этого значение опыта для следующего уровня становится больше.

Очки улучшений тратятся через функцию *SpendUpgradePoint*. Она проверяет наличие доступных очков, применяет выбранное улучшение,

уменьшает количество очков и сохраняет данные персонажа. Это не позволяет улучшать характеристики без нужного количества очков.

Доступные направления прокачки:

- увеличение максимального значения *stamina*;
- увеличение урона по руде через *OreDamage*;
- увеличение скорости движения через *WalkSpeed* и *SprintSpeed*.

На рисунке 4.26 показана логика добавления опыта и траты очков улучшений.

```
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Resources")
int32 CollectedOreResources;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Resources")
int32 CollectedGold;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Inventory")
int32 StaminaPotionCount;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 PlayerLevel;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 CurrentExperience;
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 ExperienceToNextLevel;|
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Progression")
int32 AvailableUpgradePoints;
```

Рисунок 4.26 - Добавление опыта и улучшение характеристик персонажа

Улучшение максимальной *stamina* увеличивает значение *MaxStamina*. После этого текущее значение *stamina* также восстанавливается до нового максимума. Это полезно для игрока, который часто использует *sprint*.

Улучшение урона по руде увеличивает параметр *OreDamage*. Чем выше урон, тем меньше ударов нужно для разрушения рудного объекта. Такое направление ускоряет добычу ресурсов и получение опыта.

Улучшение скорости движения увеличивает *WalkSpeed* и *SprintSpeed*. Это помогает быстрее перемещаться между рудой, торговцем и другими объектами на карте. Меню прокачки отображается в *HUD* через функцию *DrawProgressionPanel*. В нём пользователь видит уровень, опыт, очки

улучшений и характеристики персонажа. Кнопки улучшения связаны с действиями *UpgradeMaxStamina*, *UpgradeOreDamage* и *UpgradeMoveSpeed*.

На рисунке 4.27 показано меню прокачки персонажа в *HUD*.



Рисунок 4.27 - Меню прокачки персонажа в *HUD*

Прокачка связана с добычей руды. Когда руда разрушается, персонаж получает не только ресурс, но и опыт. Поэтому каждое действие приближает игрока к новому уровню и следующему улучшению.

После изменения прогресса данные сохраняются в файл *Character.json*. Благодаря этому уровень, опыт, очки улучшений и характеристики не пропадают после закрытия игры.

Таким образом, система прокачки делает персонажа развивающимся объектом. Игрок получает опыт за добычу, повышает уровень, выбирает улучшения и видит изменение характеристик в *HUD*.

4.10 Мини-задания, рабочие заказы и развитие игрового цикла

Для удобства пользователя в проекте реализована система настроек. Она позволяет изменять звук, чувствительность *camera*, масштаб меню, язык и клавиши управления. Эти параметры находятся в классе *AThiefPlayerController*, потому что именно *controller* обрабатывает меню и пользовательский *input*.

Настройки открываются через *HUD*. В стартовом *lobby* пользователь может открыть их до начала игры, а на игровой карте они доступны через меню паузы. Это позволяет менять параметры как перед запуском, так и во время игры.

Меню настроек разделено на вкладки *Audio*, *Controls* и *Interface*. Переключение активной вкладки выполняется через функцию *SetActiveSettingsTab*. Если пользователь уходит с вкладки управления, ожидание новой клавиши отменяется.

На рисунке 4.28 показана структура настроек в *HUD*.

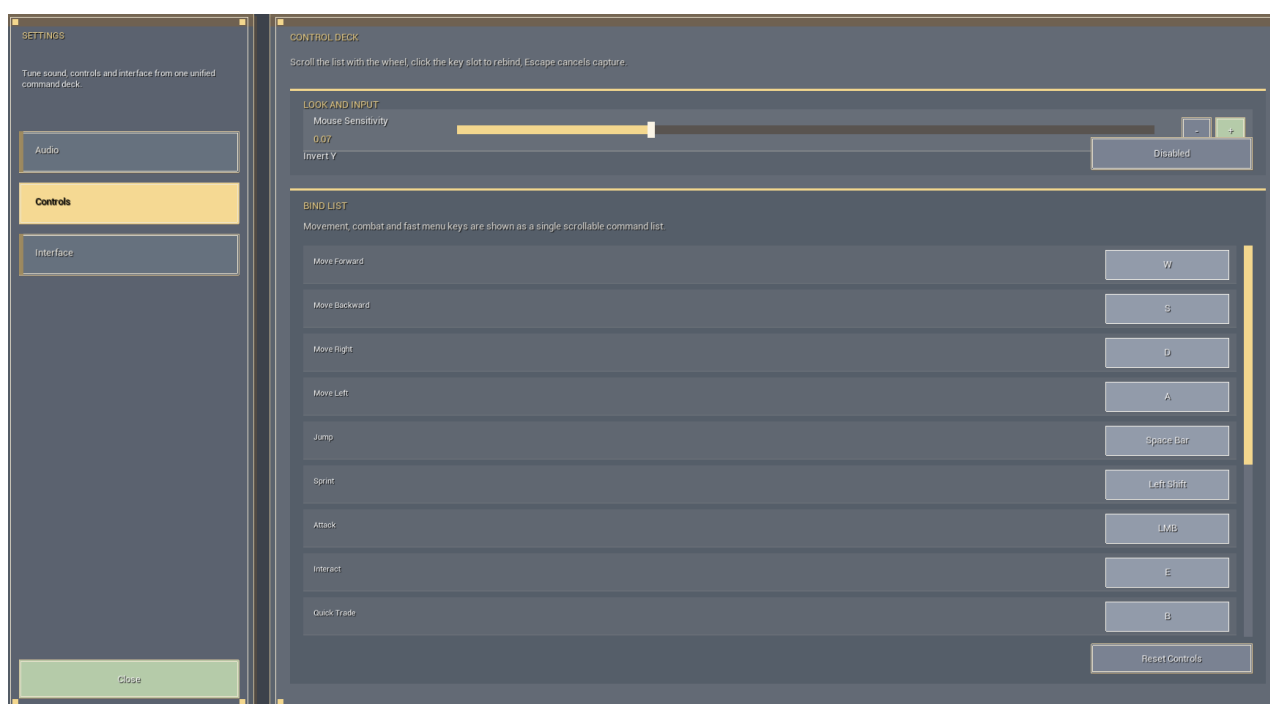


Рисунок 4.28 - Панель пользовательских настроек проекта

Звуковые настройки включают общую громкость, громкость музыки и громкость звуковых эффектов. Для них используются функции *AdjustMasterVolume*, *AdjustMusicVolume* и *AdjustSfxVolume*. Каждая функция изменяет значение в допустимых границах, применяет настройку и сохраняет данные в *Settings.json*.

Настройка чувствительности *camera* выполняется функцией *AdjustLookSensitivity*. Также есть функция *ToggleLookYInversion*, которая включает или выключает инверсию вертикального обзора. Это удобно для пользователей с разными привычками управления. Настройки *UI* включают масштаб меню и язык. Масштаб изменяется через *AdjustMenuScale*, а язык переключается функцией *CycleLanguage*. В проекте используются варианты *English* и *Russian*, а текст в *HUD* выбирается с учётом текущего языка.

Основные пользовательские настройки:

- общая громкость звука;
- громкость музыки;
- громкость звуковых эффектов;
- чувствительность *camera*;
- инверсия вертикального обзора;
- масштаб меню;
- язык отображения текста.

На рисунке 4.29 показаны функции изменения параметров настроек.

```
void AThiefPlayerController::AdjustMasterVolume(const float Delta){
    MasterVolumeSetting = FMath::Clamp(MasterVolumeSetting + Delta, 0.0f,
1.0f);
    ApplyAudioSettings();
    SaveSettingsToJson();}
void AThiefPlayerController::AdjustMenuScale(const float Delta){
    MenuScaleSetting = FMath::Clamp(MenuScaleSetting + Delta, MinMenuScale,
MaxMenuScale);
    SaveSettingsToJson();}
void AThiefPlayerController::AdjustLookSensitivity(const float Delta){
    LookSensitivitySetting = FMath::Clamp(LookSensitivitySetting + Delta,
MinLookSensitivity, MaxLookSensitivity);
    ApplyLookSettings();
    SaveSettingsToJson();}
void AThiefPlayerController::CycleLanguage(const int32 Direction){
    constexpr int32 LanguageCount = 2;
    int32 NewLanguageIndex = static_cast<int32>(CurrentLanguage) + Direction;
    while (NewLanguageIndex < 0){
        NewLanguageIndex += LanguageCount;}
    NewLanguageIndex %= LanguageCount;
    CurrentLanguage = static_cast<EGameLanguage>(NewLanguageIndex);
    SaveSettingsToJson();}
```

Рисунок 4.29 - Изменение пользовательских настроек в controller

Отдельно реализована система *input bindings*. Она позволяет пользователю переназначать клавиши управления. Для этого используется перечисление *ERemappableInputAction*, где описаны действия, доступные для изменения.

Во вкладке *Controls* отображается список действий и текущих клавиш. Когда пользователь выбирает действие, вызывается функция *BeginRebindingInput*. После этого *controller* ожидает новую клавишу и назначает её выбранному действию.

Назначение новой клавиши выполняется через функцию *ApplyBindingKey*. Она обновляет привязку и сохраняет настройки. Для возврата стандартного управления используется функция *ResetControlsToDefaults*.

К действиям, доступным для переназначения, относятся:

- движение персонажа по четырём направлениям;
- прыжок и *sprint*;
- атака и взаимодействие;
- использование зелья восстановления *stamina*;
- открытие торговли и прокачки;
- подтверждение, возврат и выбор пунктов меню.

На рисунке 4.30 показана вкладка управления и переназначения клавиш.

Система настроек связана с сохранениями. При изменении громкости, языка, масштаба меню или клавиш вызывается сохранение в *Settings.json*.

При следующем запуске проекта функция *LoadSettingsFromJson* загружает значения и применяет их.

Таким образом, пользовательские настройки делают проект удобнее и завершённое.

Игрок может настроить звук, управление, чувствительность *camera*, масштаб *UI* и язык, а система *input bindings* позволяет не ограничиваться фиксированным управлением.

```

void AThiefPlayerController::BeginRebindingInput(const ERemappableInputAction
InputAction){
    if (!bSettingsMenuOpen || !FindBindingDefinition(InputAction)){return;}
    ActiveSettingsTab = ESettingsPanelTab::Controls;
    PendingInputRebindAction = InputAction;
    bWaitingForInputRebind = true;}
void AThiefPlayerController::ResetControlsToDefaults(){
    UInputSettings* InputSettings = UInputSettings::GetInputSettings();
    if (!InputSettings){return;}
    for (const FRemappableBindingDefinition& Definition :
GetRemappableBindingDefinitions()){
        SetBindingKeyOnInputSettings(InputSettings, Definition,
Definition.DefaultKey);}
    InputSettings->ForceRebuildKeymaps();
    InputSettings->SaveKeyMappings();
    InputSettings->SaveConfig();
    CancelInputRebind();
    SaveSettingsToJson();}

```

Рисунок 4.30 - Переназначение *input bindings* в настройках проекта.

4.11 Сохранение прогресса

Последним важным этапом разработки стала реализация сохранений в формате *JSON*. Сохранения нужны для того, чтобы данные пользователя не исчезали после закрытия проекта. В текущей версии сохраняются настройки, прогресс персонажа и данные рабочих заказов.

Работа с сохранениями вынесена в отдельный вспомогательный класс *FMyProjectJsonSaveUtils*. Это позволяет не смешивать чтение и запись файлов с игровой логикой персонажа, *controller* или системой заказов.

Для настроек используется структура *FMyProjectSettingsSaveData*. В ней хранятся громкость, масштаб меню, чувствительность *camera*, инверсия вертикального обзора, выбранный язык и массив *input bindings*. Эта структура соответствует файлу *Settings.json*.

Для персонажа используется структура *FMyProjectCharacterSaveData*. В ней хранятся максимальное и текущее значение *stamina*, руда, золото, зелья, уровень, опыт, очки улучшений, урон по руде, скорости движения и общая статистика действий. Для рабочих заказов используется структура *FMyProjectWorkOrderSaveData*. Она хранит репутацию, количество

завершённых смен, последний выбранный заказ и статистику выполненных заказов. Эта структура соответствует файлу *WorkOrders.json*.

На рисунке 4.31 показаны структуры данных для сохранения настроек, персонажа и рабочих заказов.

```

USTRUCT()
struct FMyProjectSettingsSaveData{
    GENERATED_BODY()
    UPROPERTY()
    float MasterVolumeSetting = 1.0f;
    UPROPERTY()
    float MusicVolumeSetting = 0.65f;
    UPROPERTY()
    float SfxVolumeSetting = 1.0f;
    UPROPERTY()
    float MenuScaleSetting = 1.0f;
    UPROPERTY()
    float LookSensitivitySetting = 0.07f;
    UPROPERTY()
    bool bInvertLookY = false;
    UPROPERTY()
    uint8 Language = static_cast<uint8>(EGameLanguage::Russian)
    UPROPERTY()
    TArray<FMyProjectInputBindingSaveData> InputBindings;};
USTRUCT()
struct FMyProjectCharacterSaveData{
    GENERATED_BODY()
    UPROPERTY()
    float MaxStamina = 100.0f;
    UPROPERTY()
    float CurrentStamina = 100.0f;
    UPROPERTY()
    int32 CollectedOreResources = 0;
    UPROPERTY()
    int32 CollectedGold = 0;
    UPROPERTY()
    int32 StaminaPotionCount = 0;
    UPROPERTY()
    int32 PlayerLevel = 1;
    UPROPERTY()
    int32 CurrentExperience = 0;
    UPROPERTY()
    int32 ExperienceToNextLevel = 100;
    UPROPERTY()
    int32 AvailableUpgradePoints = 0;
    UPROPERTY()
    float OreDamage = 1.0f;
    UPROPERTY()
    float WalkSpeed = 450.0f;
    UPROPERTY()
    float SprintSpeed = 700.0f;};

```

Рисунок 4.31 - Структуры данных JSON-сохранений

Путь к файлам формируется внутри класса *FMyProjectJsonSaveUtils*.
Файлы *Settings.json*, *Character.json* и *WorkOrders.json* помещаются в папку

JsonSaves внутри папки *Saved*. Перед записью класс проверяет наличие нужной директории и создаёт её при необходимости.

Сохранение выполняется через преобразование структуры в строку формата *JSON*. Загрузка работает в обратную сторону: файл читается как текст, затем строка преобразуется обратно в структуру. Такой подход удобен, потому что данные можно проверить обычным текстовым редактором.

Основные функции класса *FMyProjectJsonSaveUtils*:

- *SaveSettingsData* записывает пользовательские настройки;
- *LoadSettingsData* загружает пользовательские настройки;
- *SaveCharacterData* записывает данные персонажа;
- *LoadCharacterData* загружает данные персонажа;
- *SaveWorkOrderData* записывает данные рабочих заказов;
- *LoadWorkOrderData* загружает данные рабочих заказов;
- *EnsureSaveDirectoryExists* проверяет наличие папки для сохранений.

На рисунке 4.32 показана реализация чтения и записи файлов *JSON*.

Сохранение настроек вызывается в классе *AThiefPlayerController*. Когда пользователь меняет громкость, чувствительность *camera*, язык, масштаб меню или клавиши, вызывается функция *SaveSettingsToJson*. При запуске проекта функция *LoadSettingsFromJson* загружает сохранённые значения и применяет их.

Сохранение персонажа вызывается в классе *AGameplayCharacterBase*. Когда персонаж получает опыт, продаёт руду, покупает зелье, использует зелье, улучшает характеристики или получает ресурсы, данные записываются в *Character.json*. При запуске персонажа функция *LoadCharacterDataFromJson* загружает сохранённый прогресс.

Сохранение рабочих заказов выполняется через систему *WorkOrderSubsystem*. Она использует *SaveWorkOrderData* и *LoadWorkOrderData*, чтобы хранить репутацию, выбранный заказ и данные завершённых смен.

Файл *Settings.json* содержит данные, связанные с удобством пользователя. В нём можно увидеть значения громкости, чувствительность *camera*, язык и назначенные клавиши.

```
bool FMyProjectJsonSaveUtils::SaveSettingsData(const
FMyProjectSettingsSaveData& SaveData){
    return SaveStructToJsonFile(SaveData, GetSettingsSavePath());}
bool FMyProjectJsonSaveUtils::LoadSettingsData(FMyProjectSettingsSaveData&
OutSaveData){
    return LoadStructFromJsonFile(OutSaveData, GetSettingsSavePath());}
bool FMyProjectJsonSaveUtils::SaveCharacterData(const
FMyProjectCharacterSaveData& SaveData){
    return SaveStructToJsonFile(SaveData, GetCharacterSavePath());}
bool FMyProjectJsonSaveUtils::LoadCharacterData(FMyProjectCharacterSaveData&
OutSaveData){
    return LoadStructFromJsonFile(OutSaveData, GetCharacterSavePath());}
bool FMyProjectJsonSaveUtils::SaveWorkOrderData(const
FMyProjectWorkOrderSaveData& SaveData){
    return SaveStructToJsonFile(SaveData, GetWorkOrderSavePath());}
bool FMyProjectJsonSaveUtils::LoadWorkOrderData(FMyProjectWorkOrderSaveData&
OutSaveData){
    return LoadStructFromJsonFile(OutSaveData, GetWorkOrderSavePath());}
template<typename TSaveStruct>
bool FMyProjectJsonSaveUtils::SaveStructToJsonFile(
const TSaveStruct& SaveData,
const FString& FilePath){
    if (!EnsureSaveDirectoryExists(FilePath)){return false;}
    FString JsonString;
    if (!FJsonObjectConverter::UStructToJsonObjectString(SaveData,
JsonString)){return false;}
    return FFileHelper::SaveStringToFile(JsonString, *FilePath);}
template<typename TSaveStruct>
bool FMyProjectJsonSaveUtils::LoadStructFromJsonFile(
TSaveStruct& OutSaveData,
const FString& FilePath){
    if (!FPaths::FileExists(FilePath)){return false;}
    FString JsonString;
    if (!FFileHelper::LoadFileToString(JsonString, *FilePath)){return false;}
    return FJsonObjectConverter::JsonObjectStringToUStruct(JsonString,
```

Рисунок 4.32 - Чтение и запись данных в формате JSON

Файл *WorkOrders.json* содержит данные рабочих заказов. Он нужен для сохранения прогресса смен, репутации и выбранного заказа.

Файл *Character.json* содержит данные игрового прогресса. В нём можно увидеть *MaxStamina*, *CurrentStamina*, количество руды, золота, зелий, уровень, опыт, очки улучшений, урон по руде и скорости движения.

На рисунке 4.33 показаны рабочие файлы сохранений проекта.

 Character	26.05.2026 9:11	JSON File	1 КБ
 Settings	27.04.2026 0:33	JSON File	2 КБ
 WorkOrders	26.05.2026 9:11	JSON File	1 КБ

Рисунок 4.33 - Рабочие файлы JSON-сохранений проекта

Использование *JSON* удобно для учебного проекта. Файл можно открыть обычным текстовым редактором, структура данных читается понятно, а для хранения настроек и прогресса не требуется сложная база данных. Сохранения также помогают при отладке. Если значение на экране изменилось, можно открыть файл и проверить, записалось ли оно. Если данные есть в файле, но не загружаются после перезапуска, нужно проверять функцию загрузки. В результате система *JSON*-сохранений связывает игровой процесс с постоянными данными. Пользователь может изменить настройки, добыть руду, продать её, купить зелье, повысить уровень, выполнить заказ и после следующего запуска продолжить работу с сохранённым состоянием.

5. ПРОВЕРКА РАБОТЫ ПРОГРАММЫ

Проверка работы программы выполнялась после завершения основных этапов разработки проекта *RPG FARM*. Главной задачей было убедиться, что пользователь может пройти основной игровой цикл без критических ошибок: запустить проект, открыть стартовое *lobby*, перейти на карту *Showcase*, управлять персонажем, добывать руду, взаимодействовать с торговцем, использовать *HUD*, прокачивать персонажа и сохранять прогресс. Проверка выполнялась в редакторе *Unreal Engine 5* в режиме *PIE*. Такой режим позволяет запускать игру прямо из редактора и быстро проверять результат без отдельной финальной сборки. Также выполнялась сборка проекта в *Visual Studio*, чтобы убедиться, что код на *C++* не содержит ошибок компиляции.

На рисунке 5.1 представлен запуск проекта в редакторе *Unreal Engine 5*.

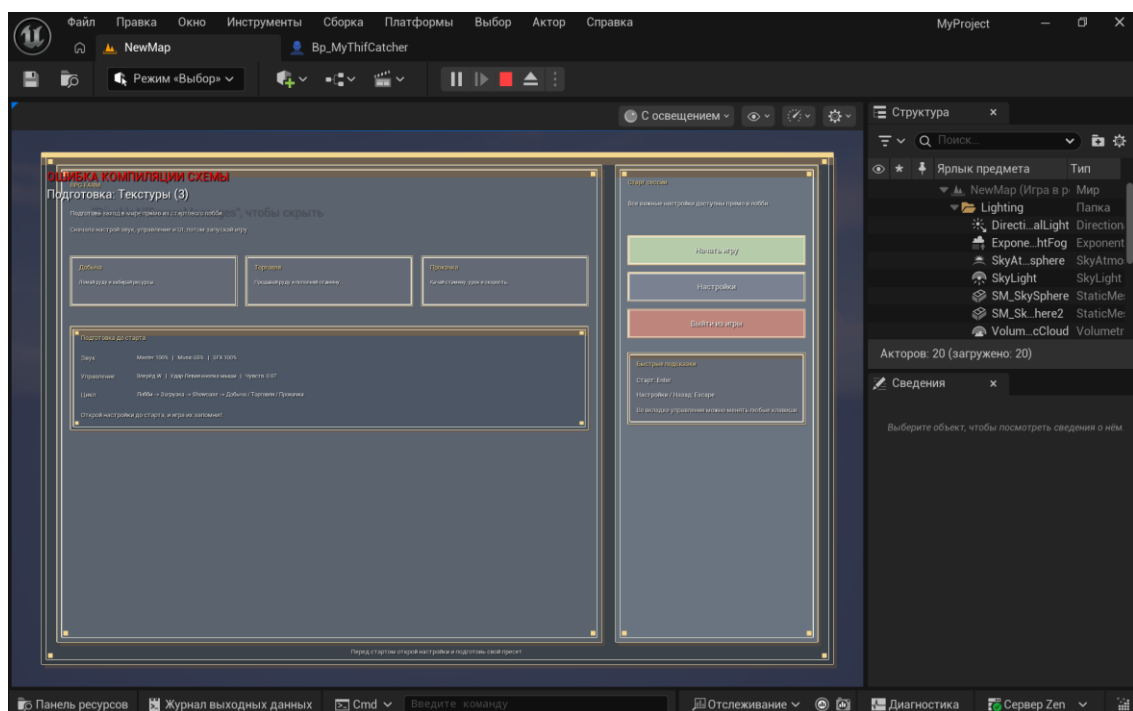


Рисунок 5.1 - Запуск проекта в редакторе *Unreal Engine 5*

При первом запуске открывается карта *NewMap*. На ней отображается стартовое *lobby* с основными кнопками. Проверялось, что меню появляется

корректно, кнопки реагируют на действия пользователя, настройки открываются без ошибок, а выход из проекта выполняется правильно. После нажатия кнопки запуска игры появляется *loading screen*, затем открывается карта *Showcase*. Во время проверки было подтверждено, что переход между картами выполняется корректно, экран загрузки отображается, а игровая сцена открывается без зависания. На рисунке 5.2 представлен экран загрузки перед переходом на игровую карту.

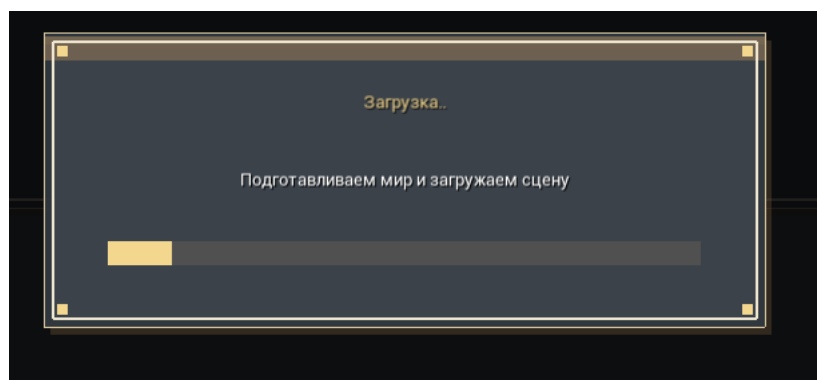


Рисунок 5.2 - Экран загрузки перед открытием карты *Showcase*

Далее проверялась работа управления персонажем. Персонаж должен перемещаться по клавишам *W*, *A*, *S*, *D*, поворачивать *camera* при движении мыши, выполнять прыжок через *Space Bar* и включать *sprint* через *Left Shift*. Во время проверки управление работало корректно. Отдельно проверялась система *stamina*. При активном *sprint* значение уменьшалось только во время движения персонажа. После остановки или отключения *sprint* запас постепенно восстанавливался. Когда *stamina* доходила до нуля, ускорение автоматически отключалось.

На рисунке 5.3 представлена проверка панели *stamina* во время игрового процесса.

Следующим этапом проверялась атака и добыча руды. Персонаж подходил к рудному объекту и выполнял удар киркой. При нажатии кнопки

атаки запускаясь *animation*, движение временно блокировалось, а в момент попадания руда получала урон.

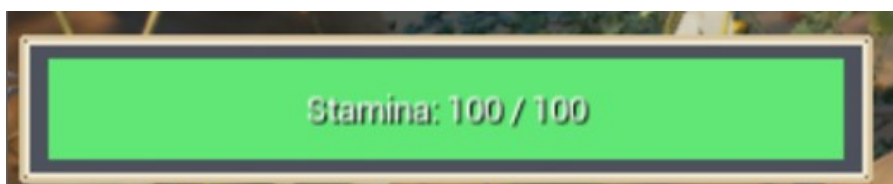


Рисунок 5.3 - Проверка отображения *stamina* в HUD

При разрушении руды объект скрывался с карты, а персонаж получал ресурс и опыт. Также проверялось, что руду нельзя добывать с большого расстояния или при неправильном направлении удара. После заданного времени рудный объект снова появлялся на карте через систему *respawn*.

На рисунке 5.4 представлена проверка атаки рудного объекта.



Рисунок 5.4 - Проверка механики атаки и добычи руды

Также проверялась панель здоровья руды. Она должна появляться только тогда, когда персонаж находится рядом с объектом. При отходе от руды панель скрывается, чтобы экран не перегружался лишними элементами.

На рисунке 5.5 представлена панель здоровья рудного объекта.

После проверки добычи выполнялась проверка торговца. Персонаж подходил к объекту торговца, после чего на экране появлялась подсказка взаимодействия. При открытии торгового меню проверялась продажа одной руды, продажа пяти единиц руды и покупка зелья восстановления *stamina*.



Рисунок 5.5 - Проверка *HealthBarWidget* рудного объекта

Во время проверки было подтверждено, что при продаже руды её количество уменьшается, а золото увеличивается. При покупке зелья золото уменьшается, а количество зелий увеличивается. Если ресурсов недостаточно, действие не выполняется. При выходе из зоны торговца меню закрывается.

На рисунке 5.6 представлена проверка взаимодействия с торговцем.

Далее проверялась работа *HUD* и игровых меню. Во время игры на экране должны отображаться ресурсы, золото, зелья, уровень, опыт и значение *stamina*. При открытии торгового меню, меню прокачки или паузы элементы *interface* не должны мешать друг другу.

Отдельно проверялась система прокачки персонажа. После разрушения руды персонаж получает опыт. При достижении нужного значения уровень повышается, а игрок получает очко улучшения. Эти очки можно потратить на максимальную *stamina*, урон по руде или скорость движения.



Рисунок 5.6 - Проверка взаимодействия с торговцем

На рисунке 5.7 представлены основные элементы *HUD* во время игры.



Рисунок 5.7 - Проверка игровых элементов *HUD*

На рисунке 5.8 представлено меню прокачки персонажа. Также проверялись пользовательские настройки. В меню изменялись громкость,

чувствительность *camera*, масштаб *UI*, язык и клавиши управления. После изменения проверялось, что параметры применяются сразу и сохраняются в *Settings.json*.

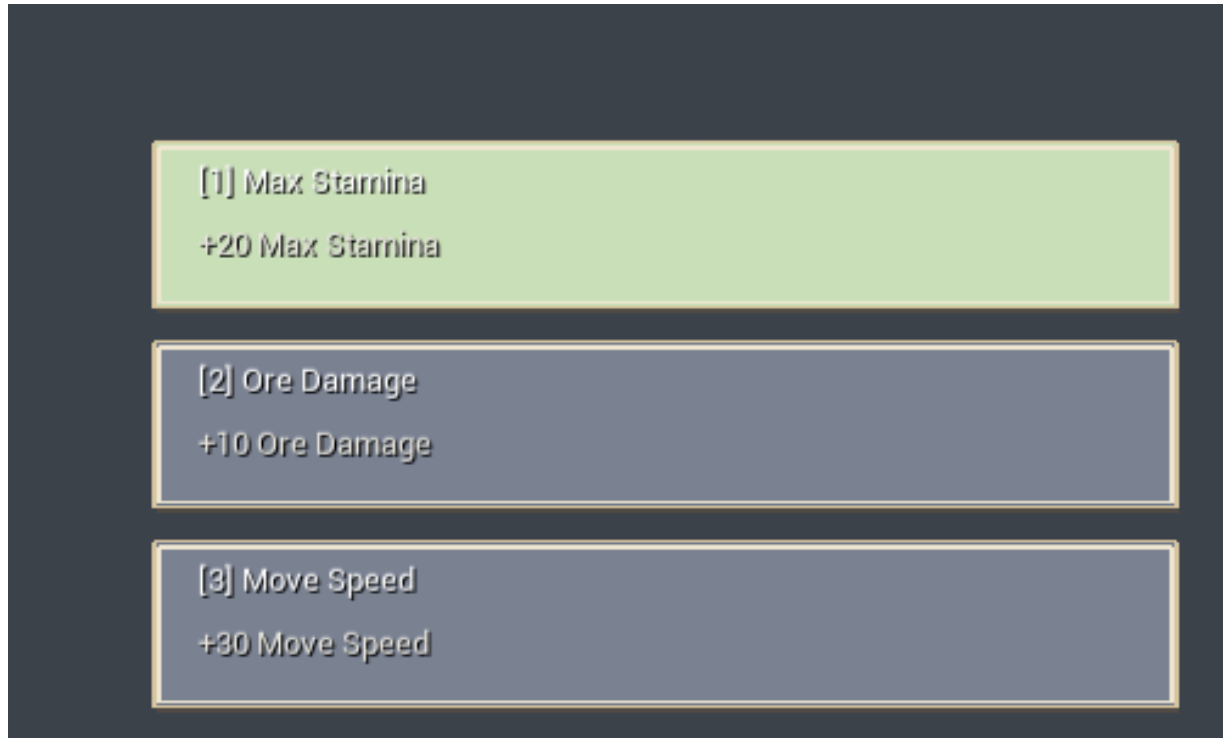


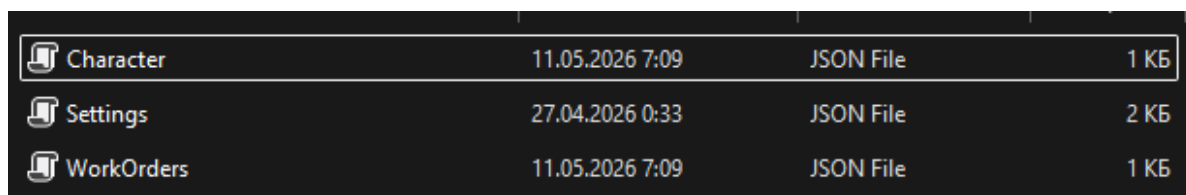
Рисунок 5.8 - Проверка меню прокачки персонажа

Последним этапом проверялись сохранения в формате *JSON*. Во время игры изменялись данные персонажа, настройки и рабочие заказы. После этого проверялись файлы *Character.json*, *Settings.json* и *WorkOrders.json*. Данные в файлах соответствовали изменениям, выполненным в игре.

После закрытия и повторного запуска проекта сохранённые значения загружались обратно. Это подтвердило, что система сохранений работает корректно и пользовательский прогресс не теряется после завершения программы.

На рисунке 5.9 представлены файлы сохранений проекта. Дополнительно выполнялась проверка сборки проекта. После изменения файлов *C++* проект собирался в *Visual Studio*. Успешная сборка подтверждала,

что классы, заголовочные файлы, зависимости и настройки проекта согласованы между собой.



 Character	11.05.2026 7:09	JSON File	1 КБ
 Settings	27.04.2026 0:33	JSON File	2 КБ
 WorkOrders	11.05.2026 7:09	JSON File	1 КБ

Рисунок 5.9 - Проверка файлов JSON-сохранений

По результатам проверки можно сделать вывод, что основные функции проекта работают корректно. Пользователь может запустить игру, перейти из *lobby* на карту *Showcase*, управлять персонажем, использовать *sprint*, добывать руду, получать ресурсы и опыт, взаимодействовать с торговцем, покупать зелья, прокачивать характеристики, изменять настройки и сохранять прогресс.

Таким образом, проект *RPG FARM* прошёл проверку основных игровых систем. Реализованные механики связаны между собой и позволяют пользователю выполнить полный цикл действий внутри игры. Это подтверждает работоспособность программы и готовность проекта к демонстрации в рамках учебной работы.

ЗАКЛЮЧЕНИЕ

В результате выполнения проекта была разработана учебная игровая система *RPG FARM* на базе *Unreal Engine 5* и языка *C++*. Цель работы заключалась в создании проекта, где пользователь может управлять персонажем, перемещаться по карте, добывать руду, взаимодействовать с торговцем, использовать *HUD*, развивать характеристики и сохранять прогресс.

В ходе разработки были реализованы стартовое *lobby*, переход на карту *Showcase*, персонаж с *camera* от третьего лица, движение, прыжок, *sprint*, система *stamina*, атака киркой, добыча руды, восстановление объектов через *respawn*, торговля, прокачка, настройки, рабочие заказы и сохранение данных в формате *JSON*.

Основная логика проекта написана на *C++*. Для персонажа используются классы *AGameplayCharacterBase* и *AThiefCatcher*, для руды создан класс *AOreStoneBase*, для торговца используется *ATraderNPC*, а экранный *interface* реализован через *APlayerGameHUD*. Такое разделение помогает сделать структуру проекта понятной и удобной для сопровождения.

Во время работы были закреплены навыки создания классов в *Unreal Engine 5*, настройки *Actor* и *Character*, обработки *input*, работы с компонентами, создания *HUD*, реализации игровых состояний, сохранения данных и проверки проекта в редакторе. Также был получен опыт организации проекта, где код, ресурсы, настройки и рабочие данные разделены по папкам.

Разработанный проект можно считать завершённой основой небольшой игры. В нём есть понятный цикл: пользователь запускает игру из *lobby*, переходит на карту, добывает руду, получает ресурсы и опыт, продаёт руду торговцу, покупает зелья восстановления *stamina*, улучшает характеристики и сохраняет прогресс.

В дальнейшем проект можно расширять. Можно добавить новые виды руды, дополнительные предметы, новые модели торговцев, улучшенную карту, визуальные эффекты, расширенную прокачку и более сложную экономику. При этом текущая версия уже демонстрирует основные возможности учебного проекта на *Unreal Engine 5* и соответствует поставленной задаче.

СПИСОК ЛИТЕРАТУРЫ

1. Epic Games. Unreal Engine Documentation. URL: <https://dev.epicgames.com/documentation>.
2. Epic Games. Programming with C++ in Unreal Engine. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine>.
3. Epic Games. Actors in Unreal Engine. URL: https://dev.epicgames.com/documentation/unreal-engine/actors-in-unreal-engine?application_version=5.7.
4. Epic Games. Setting Up a Character in Unreal Engine. URL: <https://dev.epicgames.com/documentation/es-es/unreal-engine/setting-up-a-character-in-unreal-engine>.
5. Epic Games. Unreal Motion Graphics UI Designer. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-for-unreal-engine>.
6. Epic Games. Blueprints Visual Scripting. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine>.
7. Microsoft Learn. C++ documentation. URL: <https://learn.microsoft.com/en-us/cpp/>.
8. Microsoft. Visual Studio C++ development. URL: <https://visualstudio.microsoft.com/vs/cplusplus/>.
9. cppreference.com. C++ reference. URL: <https://en.cppreference.com/cpp>.
10. JSON.org. Introducing JSON. URL: <https://www.json.org/>.
11. ISO C++ Foundation. C++ Core Guidelines. URL: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>.
12. Stroustrup B. The C++ Programming Language. 4th ed. Addison-Wesley Professional, 2013.

13. Lippman S. B., Lajoie J., Moo B. E. *C++ Primer. 5th ed. Addison-Wesley Professional*, 2012.
14. Meyers S. *Effective Modern C++*. O'Reilly Media, 2014.
15. Gregory J. *Game Engine Architecture. 3rd ed. CRC Press*, 2018.
16. Nystrom R. *Game Programming Patterns*. Genever Benning, 2014.
URL: <https://gameprogrammingpatterns.com/>.
17. Sherif W., Whittle S. *Unreal Engine 4 Scripting with C++ Cookbook*. Packt Publishing, 2016.
18. Doran J. P. *Unreal Engine 5 Game Development with C++ Scripting*. Packt Publishing, 2022.

ДОДАТОК А

Фрагменты программного кода

Фрагмент 1. Создание камеры и компонентов персонажа в классе *AThifCatcher*.

```

AThifCatcher::AThifCatcher()
: Super() {
    SpringArm =
CreateDefaultSubobject<USpringArmComponent>(TEXT("SpringArmComponent"));
    SpringArm->SetupAttachment(RootComponent);
    SpringArm->TargetArmLength = 420.0f;
    SpringArm->SetRelativeLocation(FVector(0.0f, 0.0f, 70.0f));
    SpringArm->bEnableCameraLag = true;
    SpringArm->CameraLagSpeed = 12.0f;
    SpringArm->bEnableCameraRotationLag = true;
    SpringArm->CameraRotationLagSpeed = 14.0f;
    SpringArm->bUsePawnControlRotation = true;
    Camera = CreateDefaultSubobject<UCameraComponent>(TEXT("MainCamera"));
    Camera->SetupAttachment(SpringArm);
    Camera->bUsePawnControlRotation = false;
    Camera->SetRelativeRotation(FRotator(-10.0f, 0.0f, 0.0f));
    if (UCharacterMovementComponent* MovementComponent =
GetCharacterMovement()) {
        MovementComponent->bOrientRotationToMovement = true;
        MovementComponent->RotationRate = FRotator(0.0f, 540.0f, 0.0f);
        MovementComponent->JumpZVelocity = 650.0f;
        MovementComponent->AirControl = 0.35f;
        MovementComponent->MaxWalkSpeed = WalkSpeed;}}

```

Фрагмент 2. Привязка *input* к функциям персонажа. Этот код связывает клавиши и оси управления с функциями персонажа. Через него работают движение, поворот камеры, прыжок и *sprint*.

```

void AThifCatcher::SetupPlayerInputComponent(UInputComponent*
PlayerInputComponent) {
    Super::SetupPlayerInputComponent(PlayerInputComponent);
    if (!PlayerInputComponent) {return;}
    PlayerInputComponent->BindAxis(TEXT("MoveForwardBackward"), this,
&AThifCatcher::MoveForwardBackward);
    PlayerInputComponent->BindAxis(TEXT("MoveRightLeft"), this,
&AThifCatcher::MoveRightLeft);
    PlayerInputComponent->BindAxis(TEXT("Turn"), this,
&AThifCatcher::AddControllerYawInput);
    PlayerInputComponent->BindAxis(TEXT("lookUpDown"), this,
&AThifCatcher::AddControllerPitchInput);
    PlayerInputComponent->BindAction(TEXT("Jump"), IE_Pressed, this,
&AThifCatcher::Jump);
    PlayerInputComponent->BindAction(TEXT("Jump"), IE_Released, this,
&AThifCatcher::StopJump);
    PlayerInputComponent->BindAction(TEXT("Sprint"), IE_Pressed, this,
&AThifCatcher::Sprint);
    PlayerInputComponent->BindAction(TEXT("Sprint"), IE_Released, this,
&AThifCatcher::StopSprint);}

```


Фрагмент показывает начало атаки киркой. Перед выполнением удара сначала проверяется текущее состояние персонажа, чтобы убедиться, что действие может быть выполнено корректно. Затем при необходимости отключается sprint, после чего временно блокируется движение персонажа. Завершающим этапом запускается *attack montage*, который отвечает за анимацию замаха и самого удара киркой. Такой порядок действий позволяет синхронизировать игровую логику и визуальное отображение атаки. Кроме того, это предотвращает некорректное совмещение удара с другими действиями персонажа. В результате процесс добычи ресурса выглядит более целостным и предсказуемым для игрока.

Фрагмент 3. Запуск атаки персонажа в классе *AGameplayCharacterBase*.

```
void AGameplayCharacterBase::Attack(){
    if (bIsAttacking || !bCanAttack){return;}
    if (const AThiefPlayerController* ThiefController =
Cast<AThiefPlayerController>(GetController())){
        if (ThiefController->IsAnyModalOpen()){return;}}
    bCanAttack = false;
    bIsAttacking = true;
    SetSprintActive(false);
    StopJumping();
    if (AController* CurrentController = GetController()){
        CurrentController->SetIgnoreMoveInput(true);}
    if (UCharacterMovementComponent* MovementComponent =
GetCharacterMovement()){
        MovementComponent->StopMovementImmediately();
        MovementComponent->DisableMovement();}
    if (AttackMontage){
        PlayAnimMontage(AttackMontage, 1.0f, NAME_None);}}
```